

Question 1

A small internal feed lands files in an external location with file events disabled. The team won't provision notification queues or topics, accepts slower periodic discovery, and wants Auto Loader to track processed files across runs.

Which discovery mode meets these constraints?

- A. Use a Kafka source instead, publishing one message per landed file and tracking ingestion progress through the Kafka offsets
- B. Use directory listing, leaving notification discovery turned off and keeping the same checkpoint location across every run
- C. Use classic file notifications, creating a dedicated queue and subscribing it to the storage events of the landing location
- D. Use managed file events, turning on the file event service for the external location and letting it track new files across runs

Correct answer: B

B. Use directory listing, leaving notification discovery turned off and keeping the same checkpoint location across every run

Explanation

Correct answer:

Use directory listing, leaving notification discovery turned off and keeping the same checkpoint location across every run

Why this is correct:

Directory listing periodically scans the path for new files and Auto Loader records processed-file progress in its checkpoint. It requires no notification queue or managed event service, matching the stated tolerance for slower discovery.

Why the other options are wrong:

Use classic file notifications, creating a dedicated queue and subscribing it to the storage events of the landing location: Classic notifications require the queue or topic infrastructure the team explicitly will not provision. That operational prerequisite disqualifies this discovery mode.

Use managed file events, turning on the file event service for the external location and letting it track new files across runs: This discovery choice depends on an external-location file-events service that the prompt says is unavailable. Use the mode that needs no event infrastructure.

Use a Kafka source instead, publishing one message per landed file and tracking ingestion progress through the Kafka offsets: Kafka introduces a separate message-producing system and

tracks offsets rather than using the landed files directly. No such source infrastructure is part of the requirement.

References:

1. What is Auto Loader? | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/ingestion/cloud-object-storage/auto-loader>

Question 2

The function `corp.security.region_access` accepts one **STRING** argument and returns **BOOLEAN**. An administrator must apply it as a row filter to the `region` column of `corp.silver.claims`. Both objects exist, and privileges are in place.

Which statement attaches the filter?

A. `ALTER TABLE corp.silver.claims SET ROW FILTER corp.security.region_access ON (region);`

B. `ALTER TABLE corp.silver.claims ALTER COLUMN region SET MASK corp.security.region_access;`

C. `ALTER TABLE corp.silver.claims SET ROW FILTER corp.security.region_access ON (claim_id);`

D. `SELECT * FROM corp.silver.claims WHERE corp.security.region_access(region);`

Correct answer: A

A. `ALTER TABLE corp.silver.claims SET ROW FILTER corp.security.region_access ON (region);`

Explanation

Correct answer:

`ALTER TABLE corp.silver.claims SET ROW FILTER corp.security.region_access ON (region);`

Why this is correct:

A row-filter function is evaluated for each row and receives the table column named in the `ON` list. Binding `region_access` to `region` makes its Boolean decision govern visibility based on each row's region.

Why the other options are wrong:

`ALTER TABLE corp.silver.claims SET ROW FILTER corp.security.region_access ON (claim_id);`

Binding the function to `claim_id` supplies the wrong argument. A region predicate would then evaluate claim identifiers rather than the geographic value it is meant to check.

`SELECT * FROM corp.silver.claims WHERE corp.security.region_access(region);`: A `WHERE` clause filters only this query; it does not attach an enduring table-level access policy for later readers.

`ALTER TABLE corp.silver.claims ALTER COLUMN region SET MASK`

`corp.security.region_access;`: A column mask changes the value returned for a column. Here the function returns Boolean and is intended to include or exclude rows, so the row-filter clause is the right

mechanism.

References:

1. Row filters and column masks | Databricks on Azure:

<https://learn.microsoft.com/en-us/azure/databricks/data-governance/unity-catalog/filters-and-masks>

Question 3

To cut operational effort, a team picks a Lakeflow Connect managed connector over custom notebook extraction. The connector supports the source tables and their incremental behavior; network access and authentication are approved.

Which expectation is appropriate when choosing the managed connector?

- A. The connector needs only destination permissions for these source tables, with Unity Catalog access also granting authorization inside the source system
- B. The connector supports every source object and incremental change pattern in these source systems, including ones that its documentation does not list
- C. The connector handles the supported incremental ingestion from these source tables, while the team still monitors credentials, sources, failures and freshness
- D. The connector removes the team's need to monitor ingestion freshness for these source tables, with authentication failures and outages unable to interrupt it

Correct answer: C

C. The connector handles the supported incremental ingestion from these source tables, while the team still monitors credentials, sources, failures and freshness

Explanation

Correct answer:

The connector handles the supported incremental ingestion from these source tables, while the team still monitors credentials, sources, failures and freshness

Why this is correct:

A managed connector automates supported ingestion patterns, but the team remains responsible for its operational health. Credentials expire, upstream systems change, and failures or lag can occur, so freshness and failure monitoring remain necessary.

Why the other options are wrong:

The connector removes the team's need to monitor ingestion freshness for these source tables, with authentication failures and outages unable to interrupt it: Managed ingestion does not eliminate credential expiry, source outages, or monitoring responsibilities. Automation handles supported mechanics; it cannot guarantee an uninterrupted external service.

The connector supports every source object and incremental change pattern in these source systems, including ones that its documentation does not list: Managed status does not imply universal object coverage or support for every change pattern. The connector's documented capabilities still set the boundary.

The connector needs only destination permissions for these source tables, with Unity Catalog access also granting authorization inside the source system: Destination governance cannot authenticate to or authorize access in the source platform. The approved source credentials remain part of operating the connector.

References:

1. Lakeflow Connect | Databricks:

<https://www.databricks.com/product/data-engineering/lakeflow-connect>

Question 4

The `bronze_events` table stores valid JSON in a `STRING` column named `payload`. An engineer needs just the top-level `customer_id` value as a `STRING`, without building a typed `STRUCT` for the whole payload.

Which approach extracts that field directly?

- A. Call `to_json` on `payload`, which converts the stored text into a JSON value and then returns its `customer_id` attribute as a `STRING`
- B. Call `get_json_object` on `payload` with the JSON path `'$.customer_id'`, which returns only that top-level field's value as a `STRING` for each row
- C. Call `from_json` on `payload` with a schema that describes the whole record, then select the `customer_id` field value from the resulting struct as a `STRING`
- D. Call `split` on `payload` with `'customer_id'` as the delimiter, then return the top-level text after the field name as the JSON value in a `STRING`

Correct answer: B

B. Call `get_json_object` on `payload` with the JSON path `'$.customer_id'`, which returns only that top-level field's value as a `STRING` for each row

Explanation

Correct answer:

Call `get_json_object` on `payload` with the JSON path `'$.customer_id'`, which returns only that top-level field's value as a `STRING` for each row

Why this is correct:

`get_json_object` accepts a JSON string plus a `JSONPath` expression and returns the matched field as a `STRING`. The path `$.customer_id` selects the top-level property directly, avoiding a schema for fields the engineer does not need.

Why the other options are wrong:

Call `from_json` on `payload` with a schema that describes the whole record, then select the `customer_id` field value from the resulting struct as a `STRING`: `from_json` is appropriate when the application needs a typed structured record, but parsing the whole payload adds a schema requirement beyond this single-field extraction.

Call `split` on `payload` with `'customer_id'` as the delimiter, then return the top-level text after the field name as the JSON value in a `STRING`: Text splitting is not JSON parsing: property order, whitespace, escaping, and nested content make delimiter-based extraction unreliable. It can cut the value incorrectly.

Call `to_json` on `payload`, which converts the stored text into a JSON value and then returns its `customer_id` attribute as a **STRING**: Serialization turns a structured value into JSON text; it cannot navigate the already-serialized payload to pick out a property.

References:

1. `get_json_object` function | Databricks on AWS:

https://docs.databricks.com/aws/en/sql/language-manual/functions/get_json_object

Question 5

A retailer receives several million immutable JSON files daily in cloud storage. Ingestion must track discovered files across restarts, process incrementally, and avoid repeatedly scanning an ever-growing directory. Managed file events are available.

Which design best fits these requirements?

- A.** An hourly batch JSON read over the whole source directory, which appends every record it finds to the bronze table on each run and restart
- B.** A view defined over the source files, with each downstream consumer keeping its own record of which files it has already read
- C.** Auto Loader with managed file events, which discovers new files from storage events and keeps its progress in a persistent checkpoint
- D.** COPY INTO with force enabled on the managed storage location, which reloads every file in the ever-growing directory on each run

Correct answer: C

C. Auto Loader with managed file events, which discovers new files from storage events and keeps its progress in a persistent checkpoint

Explanation

Correct answer:

Auto Loader with managed file events, which discovers new files from storage events and keeps its progress in a persistent checkpoint

Why this is correct:

Auto Loader with managed file events consumes a cache of cloud storage events to discover new files incrementally; its checkpoint preserves stream progress across restarts. This fits immutable high-volume arrivals without repeatedly listing an ever-growing directory after initial synchronization.

Why the other options are wrong:

COPY INTO with force enabled on the managed storage location, which reloads every file in the ever-growing directory on each run: Forcing COPY INTO to reload files defeats incremental discovery and can duplicate work. It is a batch copy pattern, not the checkpointed event-driven stream requested.

An hourly batch JSON read over the whole source directory, which appends every record it finds to the bronze table on each run and restart: A repeated full-directory batch read revisits old files unless extra state and deduplication are added. It also makes discovery cost grow with the directory rather than event volume.

A view defined over the source files, with each downstream consumer keeping its own record of

which files it has already read: A view over files does not itself maintain a shared exactly-once ingestion position. Independent consumers would each need their own state and could repeatedly scan old content.

References:

1. Auto Loader with file events overview:

<https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/file-events-explained>

Question 6

An engineer needs to inspect a Delta table's recent operation history, including operation types, timestamps, and available user information. The engineer wants metadata, not rows from a historical version.

Which command should be used?

- A. DESCRIBE DETAIL on the table, which lists the recent history of each version with its operation, timestamp and available user
- B. SHOW GRANTS on the table, which lists the users who changed the table along with each operation and the time that it happened
- C. A SELECT with VERSION AS OF 2, which returns the operation types, timestamp and user information recorded for that version
- D. DESCRIBE HISTORY on the table, which lists each recent version with its operation, timestamp and any available user details

Correct answer: D

D. DESCRIBE HISTORY on the table, which lists each recent version with its operation, timestamp and any available user details

Explanation

Correct answer:

DESCRIBE HISTORY on the table, which lists each recent version with its operation, timestamp and any available user details

Why this is correct:

DESCRIBE HISTORY returns Delta table version metadata, including operations, timestamps, and available user information. The question asks for audit-like history rather than reading the rows that existed in a prior snapshot.

Why the other options are wrong:

DESCRIBE DETAIL on the table, which lists the recent history of each version with its operation, timestamp and available user: DESCRIBE DETAIL reports current table properties such as location and size; it is not the version-by-version operation history.

A SELECT with VERSION AS OF 2, which returns the operation types, timestamp and user information recorded for that version: A time-travel SELECT returns historical table data, not a metadata list of all recent operations and their actors.

SHOW GRANTS on the table, which lists the users who changed the table along with each operation and the time that it happened: SHOW GRANTS displays authorization assignments, not

who performed each data-changing operation or when.

References:

1. Work with table history | Azure Databricks:

<https://learn.microsoft.com/en-us/azure/databricks/delta/history>

Question 7

An administrator drops a Unity Catalog managed Delta table after its downstream consumers have been retired. The engineer must explain the storage consequences without promising immediate physical deletion.

Which statement correctly distinguishes its lifecycle from that of an external table?

- A. Its data files stay at their external storage location after the drop, and only the table's registration in the catalog is removed
- B. DROP TABLE converts it into an external table, which keeps its catalog entry while the managed storage behind it is released
- C. Its data files are physically erased the moment DROP TABLE returns, with no retention period or recovery window of any kind afterward
- D. Databricks manages the deletion of its data files in managed storage, subject to the retention and recovery behavior that applies to managed tables

Correct answer: D

D. Databricks manages the deletion of its data files in managed storage, subject to the retention and recovery behavior that applies to managed tables

Explanation

Correct answer:

Databricks manages the deletion of its data files in managed storage, subject to the retention and recovery behavior that applies to managed tables

Why this is correct:

Unity Catalog manages the storage lifecycle for managed tables, including removal of their data files after the applicable recovery period. A drop does not mean immediate physical deletion; external-table data remains under the customer's storage lifecycle instead.

Why the other options are wrong:

Its data files stay at their external storage location after the drop, and only the table's registration in the catalog is removed: That behavior describes an external table: dropping its catalog registration leaves the customer-managed files in place. The stem explicitly identifies a managed table.

Its data files are physically erased the moment DROP TABLE returns, with no retention period or recovery window of any kind afterward: Managed-table files are retained during a recovery window and purged asynchronously, rather than erased synchronously as soon as DROP TABLE returns.

DROP TABLE converts it into an external table, which keeps its catalog entry while the managed storage behind it is released: Dropping a table removes its registration; it does not convert it into

another table type or preserve the dropped table's catalog entry.

References:

1. Object storage lifecycle in Unity Catalog:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/object-storage-lifecycle>

Question 8

A mask function must return the original `tax_id` for members of the payroll account group and the literal `REDACTED` for every other reader. `tax_id` is `STRING`, and the function is attached to a column separately.

Which statement defines the required function?

A. `CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(is_account_group_member('payroll'), 'REDACTED', tax_id);`

B. `CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(tax_id IS NULL, 'REDACTED', tax_id);`

C. `CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(is_account_group_member('payroll'), tax_id, NULL);`

D. `CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(is_account_group_member('payroll'), tax_id, 'REDACTED');`

Correct answer: D

D. `CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(is_account_group_member('payroll'), tax_id, 'REDACTED');`

Explanation

Correct answer:

`CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(is_account_group_member('payroll'), tax_id, 'REDACTED');`

Why this is correct:

A SQL mask UDF must return the original value for the authorized group and a same-type replacement for other readers. The IF condition using account-group membership expresses precisely that identity-dependent branch for a `STRING` column.

Why the other options are wrong:

`CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(is_account_group_member('payroll'), tax_id, NULL);`: Returning `NULL` hides the value but does not provide the specified literal `REDACTED`. The output would also make the intended distinction less explicit to downstream consumers.

`CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN`

IF(is_account_group_member('payroll'), 'REDACTED', tax_id);: The conditional branches are reversed, exposing the redaction string to payroll members and the original tax identifier to everyone else.

CREATE FUNCTION mask_tax(tax_id STRING) RETURNS STRING RETURN IF(tax_id IS NULL, 'REDACTED', tax_id);: Testing whether the data itself is null ignores the caller's group membership. Non-null tax identifiers would be exposed equally to every reader.

References:

1. Column mask clause | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-column-mask>

Question 9

A nightly job's run has failed. The task graph shows extract Succeeded, transform Failed, and both load_sales and load_returns Upstream failed. Analysts report that the sales summary table was not refreshed.

Where should the engineer start investigating?

- A. Start with load_sales, rerunning it on its own to refresh the sales summary table before looking at any of the other tasks
- B. Start with the cluster event log, and treat an Upstream failed status as a sign that the job's compute was lost partway through the run
- C. Start with extract, and treat the first task in the job's graph as the most likely source of the run's failure overall
- D. Start with the transform task's output and logs, and treat the Upstream failed status of both load tasks as a consequence of it

Correct answer: D

D. Start with the transform task's output and logs, and treat the Upstream failed status of both load tasks as a consequence of it

Explanation

Correct answer:

Start with the transform task's output and logs, and treat the Upstream failed status of both load tasks as a consequence of it

Why this is correct:

The task graph shows transform failed, and both load tasks are marked upstream-failed rather than having run and failed themselves. Start with the failed task's output and logs to find the root cause; downstream skips are consequences.

Why the other options are wrong:

Start with load_sales, rerunning it on its own to refresh the sales summary table before looking at any of the other tasks: The load task did not execute successfully because its dependency failed. Rerunning it independently risks using stale or incomplete transformed data and skips the actual failure investigation.

Start with extract, and treat the first task in the job's graph as the most likely source of the run's failure overall: extract succeeded, so the graph provides direct evidence against treating it as the first suspect. The failed transform is the earliest task with a failure state.

Start with the cluster event log, and treat an Upstream failed status as a sign that the job's

compute was lost partway through the run: An upstream-failed status indicates dependency failure prevented execution; it is not evidence by itself of lost compute. Inspect the actual failed task before cluster diagnostics.

References:

1. Monitor Lakeflow Jobs | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/monitor>

Question 10

A user receives **USE SCHEMA** on **corp.reporting** and already has **USE CATALOG** on **corp**. The user has no **SELECT** grants, ownership, or additional group memberships.

What does the new privilege enable by itself?

- A. Creating tables and functions anywhere in **corp.reporting**, while the objects that already exist there stay unreadable to the user
- B. Managing the permissions on every object in **corp.reporting**, including granting **SELECT** on its existing tables to other users and groups
- C. Resolving **corp.reporting** as a namespace, while reading or changing any object inside it still needs that object's own separate privilege
- D. Reading every existing and future table in **corp.reporting**, without any separate **SELECT** grant being needed on those tables or on the schema

Correct answer: C

C. Resolving **corp.reporting** as a namespace, while reading or changing any object inside it still needs that object's own separate privilege

Explanation

Correct answer:

Resolving **corp.reporting** as a namespace, while reading or changing any object inside it still needs that object's own separate privilege

Why this is correct:

USE SCHEMA allows a principal with parent **USE CATALOG** to resolve and navigate the schema namespace. Reading a table still requires **SELECT**, while creating or modifying objects requires the corresponding object-level or schema-level privilege.

Why the other options are wrong:

Reading every existing and future table in corp.reporting, without any separate SELECT grant being needed on those tables or on the schema: Namespace usage does not imply data access. Unity Catalog requires **SELECT** on the table or view in addition to the parent usage privileges.

Managing the permissions on every object in corp.reporting, including granting SELECT on its existing tables to other users and groups: Permission management requires ownership or an appropriate **MANAGE/administrative** privilege. **USE SCHEMA** only grants traversal and namespace usage.

Creating tables and functions anywhere in corp.reporting, while the objects that already exist there stay unreadable to the user: Creating tables or functions requires creation privileges such as

CREATE TABLE or CREATE FUNCTION. USE SCHEMA alone is insufficient.

References:

1. Manage schemas | Databricks on AWS:

<https://docs.databricks.com/aws/en/schemas/manage-schema>

Question 11

An engineer explains why an Auto Loader query that was stopped mid-run resumes without reloading the files it already processed. The query uses the same source, target and checkpoint location as before.

Which mechanism accounts for this behavior?

- A. Delta Lake time travel, which lets the query compare the target table's previous version with the files listed in the source directory
- B. Unity Catalog lineage, which records every source file path that has already flowed into the bronze table through the Auto Loader query
- C. The Spark RDD cache, which keeps the list of processed files in executor memory so that the list survives when the query is restarted
- D. Structured Streaming, which records the files and offsets committed by each micro-batch in the query's checkpoint location on storage

Correct answer: D

D. Structured Streaming, which records the files and offsets committed by each micro-batch in the query's checkpoint location on storage

Explanation

Correct answer:

Structured Streaming, which records the files and offsets committed by each micro-batch in the query's checkpoint location on storage

Why this is correct:

Structured Streaming checkpoints persist source offsets, committed micro-batches, and query metadata. Reusing the same checkpoint lets the restarted query resume from its saved position, so already committed files are not treated as unseen input.

Why the other options are wrong:

Delta Lake time travel, which lets the query compare the target table's previous version with the files listed in the source directory: Historical reads answer what a Delta snapshot contained at a chosen version; they do not persist a stream's next source position. Resume state comes from the checkpoint.

Unity Catalog lineage, which records every source file path that has already flowed into the bronze table through the Auto Loader query: Lineage describes data dependencies and flows for governance and impact analysis. It is not the durable operational state used to resume a stream.

The Spark RDD cache, which keeps the list of processed files in executor memory so that the list

survives when the query is restarted: Executor cache is volatile process memory and disappears when the query stops or compute restarts. Restart recovery needs persisted checkpoint state, not cached RDD data.

References:

1. Structured Streaming checkpoints:

<https://docs.databricks.com/aws/en/structured-streaming/checkpoints>

Question 12

A new engineer is documenting how a Delta table in Unity Catalog stores its data and records changes. The table uses the standard Delta format, and no special table features have been enabled.

Which TWO statements are correct?

- A. Table data is stored as Parquet data files in cloud object storage, in the same table directory as the transaction log
- B. Each write deletes every earlier version of the table, leaving only the newest set of data files in cloud storage
- C. Readers of the table are blocked as soon as a writer starts a transaction, and they resume only when that commit completes
- D. Table data is held inside the Unity Catalog metastore service itself rather than as data files in cloud object storage
- E. Each committed change is recorded as an ordered, numbered entry in the transaction log kept in the table directory

Correct answers: A, E

- A. Table data is stored as Parquet data files in cloud object storage, in the same table directory as the transaction log
- E. Each committed change is recorded as an ordered, numbered entry in the transaction log kept in the table directory

Explanation

Correct answer:

Each committed change is recorded as an ordered, numbered entry in the transaction log kept in the table directory

Table data is stored as Parquet data files in cloud object storage, in the same table directory as the transaction log

Why this is correct:

Delta stores table rows in Parquet data files in cloud storage and records committed file additions and removals in its transaction log. The ordered log versions let readers reconstruct a consistent table snapshot without replacing the table with only its latest files.

Why the other options are wrong:

Each write deletes every earlier version of the table, leaving only the newest set of data files in cloud storage: Old files can remain available for time travel and concurrent readers until retention

cleanup makes them eligible for vacuuming. A commit adds a new version; it does not erase all earlier versions.

Readers of the table are blocked as soon as a writer starts a transaction, and they resume only when that commit completes: Delta uses optimistic concurrency and snapshot isolation so readers can continue reading a consistent snapshot during a write. The transaction log coordinates commits instead of blocking all readers at writer start.

Table data is held inside the Unity Catalog metastore service itself rather than as data files in cloud object storage: Unity Catalog stores metadata and governance information, while the table's Parquet data resides in object storage. The metastore is not the storage container for the row payload.

References:

1. Frequently asked questions (FAQ) | Delta Lake: <https://docs.delta.io/delta-faq/>

Question 13

The dictionary rules holds several named SQL predicates. No input row may be discarded, but violations of each predicate must be tracked.

```
from pyspark import pipelines as dp

@dp.table
def silver_events():
    return spark.readStream.table("bronze_events")
```

Which decorator belongs in the blank?

- A. `@dp.expect_all_or_fail(rules)`, which stops the table update when any input row breaks a rule, keeps no rows retained, and records every rule that failed
- B. `@dp.expect_all_or_drop(rules)`, which drops each input row that breaks any of the named rules and records the violation counts per rule
- C. `@dp.expect_all(rules)`, which keeps every input row in the target table and records the violation counts for each of the named rules
- D. `@dp.expect_or_drop` with a single rule on `event_id`, which drops rows with a null `event_id` and records violations for that one rule only

Correct answer: C

C. `@dp.expect_all(rules)`, which keeps every input row in the target table and records the violation counts for each of the named rules

Explanation

Correct answer:

`@dp.expect_all(rules)`, which keeps every input row in the target table and records the violation counts for each of the named rules

Why this is correct:

`expect_all` applies every named expectation in warning mode: violations are measured, but the rows remain in the target. That is the required policy because the stem prohibits discarding rows while still asking for per-rule tracking.

Why the other options are wrong:

`@dp.expect_all_or_drop(rules)`, which drops each input row that breaks any of the named rules and records the violation counts per rule: The `_or_drop` variant explicitly filters violating rows from the output. Recording metrics does not undo that removal, which conflicts with the no-discard constraint.

@dp.expect_or_drop with a single rule on event_id, which drops rows with a null event_id and records violations for that one rule only: The single-rule decorator evaluates only one condition and drops its failing records. The requested multi-rule policy must keep all rows and measure every named rule.

@dp.expect_all_or_fail(rules), which stops the table update when any input row breaks a rule, keeps no rows retained, and records every rule that failed: The _or_fail variant fails the flow when violations occur. That prevents the otherwise successful update instead of retaining the input records.

References:

1. Expectation recommendations and advanced patterns:

<https://docs.databricks.com/aws/en/ldp/expectation-patterns>

Question 14

A team runs an ingestion notebook, a declarative pipeline and a SQL refresh by hand each night, in that order. They want one scheduled production workflow with task dependencies, retries and a run history they can review.

What should the team build?

- A. A Delta Lake table property on the final table that refreshes its upstream notebook and pipeline in order, with retries, before each read
- B. A Databricks Connect script on a laptop that calls the three assets in order, retries any failures and writes each outcome to a local log file
- C. A Lakeflow Job that chains notebook, pipeline and SQL tasks by dependencies and adds a nightly schedule with retries on each of the three tasks
- D. A Unity Catalog lineage graph linking the notebook, pipeline and SQL assets, refreshing each task's asset with retries when upstream dependencies change

Correct answer: C

C. A Lakeflow Job that chains notebook, pipeline and SQL tasks by dependencies and adds a nightly schedule with retries on each of the three tasks

Explanation

Correct answer:

A Lakeflow Job that chains notebook, pipeline and SQL tasks by dependencies and adds a nightly schedule with retries on each of the three tasks

Why this is correct:

Lakeflow Jobs orchestrates heterogeneous work as task nodes with dependency edges, retries, schedules, and run history. A job can therefore represent the manual notebook-pipeline-SQL sequence as one observable, scheduled production workflow.

Why the other options are wrong:

A Unity Catalog lineage graph linking the notebook, pipeline and SQL assets, refreshing each task's asset with retries when upstream dependencies change: Lineage explains data dependencies but is not an execution scheduler. It will not run these assets in order or retry failed tasks.

A Databricks Connect script on a laptop that calls the three assets in order, retries any failures and writes each outcome to a local log file: A local script puts orchestration and logs outside the Databricks job graph and history. It also introduces a laptop as a production scheduler.

A Delta Lake table property on the final table that refreshes its upstream notebook and pipeline in order, with retries, before each read: Table properties configure storage or table behavior, not the

execution order of external notebooks and pipeline refreshes. A workflow scheduler must coordinate those tasks.

References:

1. Configure and edit tasks in Lakeflow Jobs | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/configure-task>

Question 15

A data engineering group already has **USE CATALOG** and **USE SCHEMA** on the parent objects of a Delta table, plus **SELECT** on the table. An administrator grants the group **MODIFY** on the table and nothing else.

Which **TWO** operations does the **MODIFY** privilege authorize on the table?

- A. Granting **SELECT** on the table to another group that already holds the usage privileges on the table's catalog and schema
- B. Deleting rows from the table with a **DELETE** statement whose predicate matches some of the rows that the table currently holds
- C. Creating a new schema in the same catalog to hold a test copy of the rows that the table currently contains
- D. Transferring ownership of the table to another group with an **ALTER TABLE** statement that includes an **OWNER TO** clause
- E. Inserting new rows into the table with an **INSERT** statement or with an append-mode write from a Spark DataFrame

Correct answers: B, E

- B. Deleting rows from the table with a **DELETE** statement whose predicate matches some of the rows that the table currently holds
- E. Inserting new rows into the table with an **INSERT** statement or with an append-mode write from a Spark DataFrame

Explanation

Correct answer

Deleting rows from the table with a **DELETE** statement whose predicate matches some of the rows that the table currently holds

Inserting new rows into the table with an **INSERT** statement or with an append-mode write from a Spark DataFrame

Why this is correct

MODIFY authorizes changing a table's data, including inserts and deletes. Unity Catalog also requires **SELECT** on the table and the parent usage privileges alongside it, and the group already holds all three. It does not itself grant authority to grant access to others, transfer ownership, or create schemas.

Why the other options are wrong

Granting **SELECT** on the table to another group that already holds the usage privileges on the

table's catalog and schema - Granting a privilege to another principal is access administration, not a DML change to the table's rows. MODIFY does not confer grant authority.

Transferring ownership of the table to another group with an ALTER TABLE statement that includes an OWNER TO clause - Ownership transfer is an administrative ownership operation, not table data modification. The question grants no ownership or separate permission-management privilege.

Creating a new schema in the same catalog to hold a test copy of the rows that the table currently contains - Creating a schema is a catalog-level namespace operation and requires schema-creation authority. A table's MODIFY privilege does not extend to its parent catalog.

References

- [Unity Catalog privileges reference | Azure Databricks](#)

Question 16

A finance report must show sales as of midnight and be delivered by 06:30 each day. Another team's pipeline updates the sales table many times overnight, and the final update can land shortly after midnight.

Which trigger fits the report job?

- A. A continuous trigger, which keeps a run active through the night and starts the next run as soon as the previous one has ended
- B. A table update trigger on the sales table, which starts the report job each time the upstream pipeline commits new rows during the night
- C. A scheduled trigger at 06:00 in the finance team's time zone, which runs the report once after the overnight updates have all landed
- D. A file arrival trigger on the sales table's storage path, which starts the report job whenever new data files appear in that location

Correct answer: C

C. A scheduled trigger at 06:00 in the finance team's time zone, which runs the report once after the overnight updates have all landed

Explanation

Correct answer:

A scheduled trigger at 06:00 in the finance team's time zone, which runs the report once after the overnight updates have all landed

Why this is correct:

A scheduled trigger runs once at a chosen time rather than on every upstream table update. Scheduling at 06:00 leaves a buffer after the possible shortly-after-midnight final refresh and still precedes the 06:30 delivery deadline.

Why the other options are wrong:

A table update trigger on the sales table, which starts the report job each time the upstream pipeline commits new rows during the night: Multiple overnight commits can launch multiple report runs, including before the final refresh. This trigger follows changes rather than the required once-daily delivery time.

A continuous trigger, which keeps a run active through the night and starts the next run as soon as the previous one has ended: A continuous trigger is designed to keep work running by launching another run after completion. It does not wait until the daily reporting deadline.

A file arrival trigger on the sales table's storage path, which starts the report job whenever new

data files appear in that location: A file-arrival trigger watches a Unity Catalog storage location, not the semantic completion of table refreshes. New data files may also arrive before the final table update.

References:

1. Lakeflow Jobs | Databricks on AWS: <https://docs.databricks.com/aws/en/jobs>

Question 17

An engineer developing a Python ETL notebook must run selected cells repeatedly and inspect intermediate DataFrames. Only classic compute is permitted, and the notebook is not ready for unattended production runs.

Which compute resource best supports this development workflow?

- A. An instance pool attached directly to the notebook, which runs each cell on idle pool instances without creating a cluster first
- B. A Pro SQL warehouse attached to the notebook, with each Python transformation rewritten as a SQL query that runs on a dashboard
- C. An all-purpose cluster attached to the notebook, kept running for the session and set to terminate after a period of inactivity
- D. A job cluster created for a scheduled task, with each code change checked in the run output after the job run has completed

Correct answer: C

C. An all-purpose cluster attached to the notebook, kept running for the session and set to terminate after a period of inactivity

Explanation

Correct answer:

An all-purpose cluster attached to the notebook, kept running for the session and set to terminate after a period of inactivity

Why this is correct:

All-purpose compute supports interactive notebook development: the engineer can execute selected cells, inspect results, and reuse the attached session. Auto-termination controls idle resource use, while a job cluster is optimized for unattended task runs rather than iterative exploration.

Why the other options are wrong:

A job cluster created for a scheduled task, with each code change checked in the run output after the job run has completed: Jobs compute is provisioned for workflow tasks and typically ends with the run, making it poorly suited to a developer's repeated interactive cell execution and inspection.

A Pro SQL warehouse attached to the notebook, with each Python transformation rewritten as a SQL query that runs on a dashboard: A SQL warehouse is designed for SQL analytics, not arbitrary Python ETL cells. Rewriting the notebook into SQL changes the work instead of supplying suitable compute.

An instance pool attached directly to the notebook, which runs each cell on idle pool instances

without creating a cluster first: An instance pool is a source of ready instances that can speed cluster startup; it is not itself a Spark cluster to which a notebook attaches.

References:

1. **Notebook compute resources:** <https://docs.databricks.com/aws/en/notebooks/notebook-compute>

Question 18

A source overwrites JSON files in place, and Auto Loader reprocesses them, producing repeated business keys. Replacements may carry higher or lower revisions than stored records. Silver must keep exactly the highest revision per key, including corrected values.

Which downstream change meets this requirement?

- A. MERGE by business key after keeping each key's top revision in the batch, updating a stored match only when that revision is higher
- B. Deduplicate by business key before writing, keeping whichever row for each key happens to arrive first in the micro-batch
- C. MERGE by business key, replacing every matched row with the incoming record regardless of which revision number it carries
- D. Deduplicate across all columns before writing, keeping every higher and lower revision of each business key in silver

Correct answer: A

A. MERGE by business key after keeping each key's top revision in the batch, updating a stored match only when that revision is higher

Explanation

Correct answer:

MERGE by business key after keeping each key's top revision in the batch, updating a stored match only when that revision is higher

Why this is correct:

Keeping the highest revision per key in the incoming batch avoids ambiguous multiple source matches, and a conditional MERGE updates an existing target only when the chosen revision is higher. The same merge can insert a key not yet present, preserving corrected values without rolling back newer state.

Why the other options are wrong:

Deduplicate by business key before writing, keeping whichever row for each key happens to arrive first in the micro-batch: Keeping the first arrival makes the outcome depend on file or micro-batch ordering, not revision precedence. A later corrected record may be discarded even when it carries the highest revision.

MERGE by business key, replacing every matched row with the incoming record regardless of which revision number it carries: Unconditional replacement lets a stale lower revision overwrite a newer stored value. Matching by key identifies the record, but the revision comparison decides whether the update is valid.

Deduplicate across all columns before writing, keeping every higher and lower revision of each business key in silver: Full-row deduplication only removes identical records; it leaves multiple revisions for the same business key. Silver would then violate its one-current-record-per-key requirement.

References:

1. Upsert into a Delta Lake table using merge: <https://docs.databricks.com/gcp/en/delta/merge>

Question 19

An auditor asks who granted **SELECT** on **finance.gold.payroll** to the **contractors** group last week, and when. The current grants are correct, and Unity Catalog system tables are enabled for the account.

Where should the engineer look?

- A. **SHOW GRANTS** on the table, reading the **contractors** group's grant along with the user who created it and the time that it was made
- B. **DESCRIBE HISTORY** on the table, finding the table version that recorded the grant to the **contractors** group along with its user and timestamp
- C. The **system.access.audit** table, filtered to the permission-change events recorded for that table during last week's dates
- D. The table's lineage graph in Catalog Explorer, finding the upstream event last week that introduced the **contractors** group's grant on the table

Correct answer: C

C. The **system.access.audit** table, filtered to the permission-change events recorded for that table during last week's dates

Explanation

Correct answer:

The **system.access.audit** table, filtered to the permission-change events recorded for that table during last week's dates

Why this is correct:

The account's **system.access.audit** system table contains audit events, including permission changes, with event identity and time metadata. Filtering relevant events to the table and date window can identify who granted **SELECT** and when.

Why the other options are wrong:

SHOW GRANTS on the table, reading the **contractors** group's grant along with the user who created it and the time that it was made: **SHOW GRANTS** reports the current authorization state; it does not provide a historical actor and timestamp for each grant operation.

DESCRIBE HISTORY on the table, finding the table version that recorded the grant to the **contractors** group along with its user and timestamp: Delta history records table-data operations, while Unity Catalog grant changes are authorization audit events and do not create row-data versions.

The table's lineage graph in Catalog Explorer, finding the upstream event last week that introduced the **contractors** group's grant on the table: Lineage tracks reads and writes between data

assets. A privilege grant is a governance change, not a data-flow event.

References:

1. Audit log system table reference | Microsoft Learn:

<https://learn.microsoft.com/en-us/azure/databricks/admin/system-tables/audit-logs>

Question 20

In this join stage, a few partitions hold many records for one key.

Metric	Median	Maximum
Duration	8 seconds	190 seconds
Shuffle read	24 MB	1.8 GB

AQE is on, but skew-join handling is off. Which change directly targets the uneven partitions?

- A. Reduce `spark.sql.shuffle.partitions` to pack the skewed join keys into fewer tasks, then remeasure the join stage's task durations and shuffle reads
- B. Increase the driver's memory to give the executors room for larger partitions, then remeasure the stage's task durations and shuffle reads
- C. Enable AQE skew-join handling so Spark can split the oversized join partitions into smaller tasks, then remeasure the stage's task durations and shuffle reads
- D. Disable AQE and its skew-join handling to keep the uneven partition layout chosen during planning, then remeasure the stage's task durations and shuffle reads

Correct answer: C

C. Enable AQE skew-join handling so Spark can split the oversized join partitions into smaller tasks, then remeasure the stage's task durations and shuffle reads

Explanation

Correct answer:

Enable AQE skew-join handling so Spark can split the oversized join partitions into smaller tasks, then remeasure the stage's task durations and shuffle reads

Why this is correct:

AQE skew-join handling detects oversized shuffle partitions and splits them into smaller pieces to reduce straggler tasks. The median-versus-maximum duration and shuffle-read gap is characteristic of skew, so enabling that specific adaptive rule addresses the observed bottleneck.

Why the other options are wrong:

Reduce `spark.sql.shuffle.partitions` to pack the skewed join keys into fewer tasks, then remeasure the join stage's task durations and shuffle reads: Reducing shuffle partitions makes each partition larger on average and can concentrate skewed keys further. It changes overall partitioning rather than splitting the oversized skew partition.

Increase the driver's memory to give the executors room for larger partitions, then remeasure the stage's task durations and shuffle reads: The driver coordinates planning and execution; adding its

memory does not divide skewed executor-side shuffle partitions. The measured outlier is in distributed task work.

Disable AQE and its skew-join handling to keep the uneven partition layout chosen during planning, then remeasure the stage's task durations and shuffle reads: Disabling AQE removes adaptive skew handling instead of enabling the needed behavior. A static plan cannot react to the extreme partition sizes observed during execution.

References:

1. Performance Tuning | Apache Spark: <https://spark.apache.org/docs/latest/sql-performance-tuning>

Question 21

A bundle's prod target was deployed earlier today and has not changed since. An operator must start the deployed job resource keyed reconcile now, without performing a new deployment.

Which Databricks CLI command should the operator run?

- A. databricks bundle run with the prod target and the reconcile key, which starts the already deployed job without redeploying the bundle first
- B. databricks bundle init with reconcile as the template name, which creates a project that contains the job and then runs it once
- C. databricks bundle deploy with the prod target, which redeploys the already deployed bundle and then starts reconcile as its final step
- D. databricks bundle validate with the prod target, which checks the deployed reconcile job's configuration without deploying and then starts one run

Correct answer: A

A. databricks bundle run with the prod target and the reconcile key, which starts the already deployed job without redeploying the bundle first

Explanation

Correct answer:

databricks bundle run with the prod target and the reconcile key, which starts the already deployed job without redeploying the bundle first

Why this is correct:

bundle run starts a resource already represented in the deployed bundle and accepts its resource key. Supplying the prod target selects the deployed production instance; a new deployment is unnecessary when configuration has not changed.

Why the other options are wrong:

databricks bundle validate with the prod target, which checks the deployed reconcile job's configuration without deploying and then starts one run: Validation verifies bundle configuration and does not start the job. A validate command is a preflight check, not a run request.

databricks bundle init with reconcile as the template name, which creates a project that contains the job and then runs it once: Project initialization creates bundle files locally; it does not address the already-deployed production job or start its execution.

databricks bundle deploy with the prod target, which redeploys the already deployed bundle and then starts reconcile as its final step: bundle deploy applies configuration but does not automatically launch the resource's job run. The operator needs the run command for execution.

References:

1. Deploy bundles and run workflows from the workspace:

<https://docs.databricks.com/aws/en/dev-tools/bundles/workspace-deploy>

Question 22

A 900 GB fact table joins a 60 MB dimension table. Statistics are current, but the initial plan uses a sort-merge join. The planner should choose a broadcast join for tables up to 100 MB before execution.

What should the team change?

- A. Raise `spark.default.parallelism` for the cluster, which lets the initial plan broadcast the dimension table to the executors
- B. Raise `spark.sql.autoBroadcastJoinThreshold` to 100 MB in bytes, which lets the initial plan broadcast the dimension table to the executors
- C. Lower `spark.sql.shuffle.partitions` for this query, which lets the initial plan broadcast the dimension table to the executors
- D. Raise `spark.executor.memory` on every worker node, which lets the initial plan broadcast the dimension table to the executors

Correct answer: B

B. Raise `spark.sql.autoBroadcastJoinThreshold` to 100 MB in bytes, which lets the initial plan broadcast the dimension table to the executors

Explanation

Correct answer:

Raise `spark.sql.autoBroadcastJoinThreshold` to 100 MB in bytes, which lets the initial plan broadcast the dimension table to the executors

Why this is correct:

The static auto-broadcast threshold determines whether the planner can choose to broadcast a small relation before execution. Raising it to 100 MB in bytes places the 60 MB dimension beneath the configured cutoff, enabling the desired initial physical plan.

Why the other options are wrong:

Raise `spark.executor.memory` on every worker node, which lets the initial plan broadcast the dimension table to the executors: Executor memory affects resource capacity after the plan is chosen; it does not change the planner's size threshold for selecting broadcast joins.

Lower `spark.sql.shuffle.partitions` for this query, which lets the initial plan broadcast the dimension table to the executors: Shuffle partition count controls the parallelism of shuffle stages. It does not make the planner treat a 60 MB relation as eligible for broadcast.

Raise `spark.default.parallelism` for the cluster, which lets the initial plan broadcast the dimension table to the executors: Default parallelism sets default RDD task counts, not join strategy selection. The

relevant planning control is the broadcast threshold.

References:

1. SQL performance tuning | Apache Spark:

<https://github.com/apache/spark/blob/master/docs/sql-performance-tuning.md>

Question 23

An engineer loads semicolon-delimited CSV files with a header row into an existing Unity Catalog Delta table. The source columns match the target schema.

```
COPY INTO corp.bronze.vendor_codes  
FROM '/Volumes/corp/landing/vendor_files/codes/'  
_____;
```

Which clause completes the statement?

- A. `FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'true', 'delimiter' = ';')`
- B. `FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'false', 'delimiter' = ';')`
- C. `FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'true', 'delimiter' = ',')`
- D. `FILEFORMAT = CSV COPY_OPTIONS ('header' = 'true', 'delimiter' = ';')`

Correct answer: A

A. `FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'true', 'delimiter' = ';')`

Explanation

Correct answer:

`FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'true', 'delimiter' = ';')`

Why this is correct:

`FORMAT_OPTIONS` passes CSV reader settings to `COPY INTO`. Setting the header option to true tells the parser to use the first line for column names, while the semicolon delimiter matches the source file's field separator.

Why the other options are wrong:

`FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'false', 'delimiter' = ';')`: With header=false, the first row is read as data rather than treated as column names. That would introduce header text into the target rows.

`FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'true', 'delimiter' = ',')`: A comma delimiter would parse the stated semicolon-separated records incorrectly, often leaving the whole line in one field instead of matching the target columns.

FILEFORMAT = CSV COPY_OPTIONS ('header' = 'true', 'delimiter' = ';'); COPY_OPTIONS controls copy behavior such as file handling, not source-format parsing options. Header and delimiter belong in FORMAT_OPTIONS.

References:

1. COPY INTO | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/sql/language-manual/delta-copy-into>

Question 24

A Python notebook task calculates a scalar named `cutoff_id`. A downstream task in the same run must reference it through `{{tasks.find_cutoff.values.cutoff_id}}`. The producing task key is `find_cutoff`.

Which call publishes the value under the required key?

- A. `dbutils.widgets.text` with the name `cutoff_id` and the computed value, which exposes it as a parameter to the downstream task in the same run
- B. `dbutils.jobs.taskValues.set` with the key `cutoff_id` and the computed value, called inside the notebook of the `find_cutoff` task during the run
- C. `spark.conf.set` with the key `cutoff_id` and the computed value, which shares it with the downstream task in the same run through the Spark session
- D. `dbutils.notebook.exit` with the computed value, which publishes it as `cutoff_id` for every task in the same run that depends on `find_cutoff`

Correct answer: B

B. `dbutils.jobs.taskValues.set` with the key `cutoff_id` and the computed value, called inside the notebook of the `find_cutoff` task during the run

Explanation

Correct answer:

`dbutils.jobs.taskValues.set` with the key `cutoff_id` and the computed value, called inside the notebook of the `find_cutoff` task during the run

Why this is correct:

`dbutils.jobs.taskValues.set` publishes a named value from the current task during a job run. The downstream reference combines the producing task key `find_cutoff` with the value key `cutoff_id`, so both names must match.

Why the other options are wrong:

`spark.conf.set` with the key `cutoff_id` and the computed value, which shares it with the downstream task in the same run through the Spark session: Spark configuration is session or application configuration, not the task-value channel used for cross-task dynamic references. A downstream task may run in a different session.

`dbutils.notebook.exit` with the computed value, which publishes it as `cutoff_id` for every task in the same run that depends on `find_cutoff`: `notebook.exit` returns a task's result or termination value; it does not publish an arbitrary named task value under the dynamic-reference namespace.

`dbutils.widgets.text` with the name `cutoff_id` and the computed value, which exposes it as a

parameter to the downstream task in the same run: Widgets expose notebook parameters but do not publish values from an upstream task into the job's task-value namespace.

References:

1. Databricks Utilities (dbutils) reference:

<https://docs.databricks.com/aws/en/dev-tools/databricks-utils>

Question 25

Frequent small appends left a Delta table with thousands of files under 1 MB, and scans are dominated by file opening. The team wants fewer, larger files without changing rows or discarding time-travel history.

Which maintenance action fits?

- A. Run VACUUM with the default retention period, which clears the small data files out of storage and keeps only the larger files in the table
- B. Run a DELETE whose predicate matches no rows, which makes Delta rewrite the table's existing small files into fewer and larger data files
- C. Run OPTIMIZE on the table, which rewrites its many small data files into fewer larger files while the logical rows and the version history stay the same
- D. Run RESTORE to the table's latest version, which rebuilds the data files from the transaction log in a compacted layout with larger files

Correct answer: C

C. Run OPTIMIZE on the table, which rewrites its many small data files into fewer larger files while the logical rows and the version history stay the same

Explanation

Correct answer:

Run OPTIMIZE on the table, which rewrites its many small data files into fewer larger files while the logical rows and the version history stay the same

Why this is correct:

OPTIMIZE compacts eligible Delta data files by rewriting them into fewer, larger files. Compaction changes the physical layout, while Delta's transaction log preserves the table's logical contents and prior versions for time travel; VACUUM instead removes obsolete files.

Why the other options are wrong:

Run a DELETE whose predicate matches no rows, which makes Delta rewrite the table's existing small files into fewer and larger data files: No rows qualify for deletion, so this statement has no file-rewrite work that would compact the table. Delta exposes OPTIMIZE for deliberate file reorganization.

Run VACUUM with the default retention period, which clears the small data files out of storage and keeps only the larger files in the table: Removing obsolete files is storage cleanup, not compaction of currently active data files. Reducing the retention period could also sacrifice the requested historical recovery.

Run RESTORE to the table's latest version, which rebuilds the data files from the transaction log

in a compacted layout with larger files: RESTORE creates a new version whose logical contents match an earlier version. It is a recovery operation, not a file-compaction command for the current layout.

References:

1. Compact small files in the Delta table with optimize: <https://github.com/delta-io/delta-rs/blob/main/docs/usage/optimize/small-file-compaction-with-optimize.md>

Question 26

Contract PDFs arrive in a SharePoint document library throughout the day. A downstream team extracts text later, so each file's raw bytes and metadata must reach a Unity Catalog Delta table as files arrive, without custom extraction code.

Which approach fits?

- A. A foreign catalog over SharePoint in Unity Catalog through Lakehouse Federation, which lets the team query the PDF content in place when needed
- B. An Auto Loader stream in JSON format on the SharePoint library URL, which incrementally parses each new PDF's text and metadata into catalog table columns
- C. A Lakeflow Connect SharePoint ingestion pipeline, which incrementally lands each new file's binary content and metadata in a Unity Catalog Delta table
- D. A COPY INTO job with the BINARYFILE format pointed at the SharePoint site, which reloads every file in the library into a Delta table on each run

Correct answer: C

C. A Lakeflow Connect SharePoint ingestion pipeline, which incrementally lands each new file's binary content and metadata in a Unity Catalog Delta table

Explanation

Correct answer:

A Lakeflow Connect SharePoint ingestion pipeline, which incrementally lands each new file's binary content and metadata in a Unity Catalog Delta table

Why this is correct:

The managed SharePoint connector supports incremental ingestion of files into Databricks, including unstructured content captured as binary data and associated metadata. That matches the source, format, and requirement to defer text extraction to a downstream process.

Why the other options are wrong:

A foreign catalog over SharePoint in Unity Catalog through Lakehouse Federation, which lets the team query the PDF content in place when needed: A foreign catalog is for querying supported structured external sources in place; it does not land each PDF's raw bytes and metadata in a Delta table.

An Auto Loader stream in JSON format on the SharePoint library URL, which incrementally parses each new PDF's text and metadata into catalog table columns: Auto Loader reads supported cloud-storage paths and JSON parsing expects JSON records. A SharePoint PDF is binary document content, not JSON text to parse into fields.

A COPY INTO job with the BINARYFILE format pointed at the SharePoint site, which reloads every file in the library into a Delta table on each run: COPY INTO targets supported storage paths and is a batch file-copy workflow, not a managed incremental SharePoint connector with this source integration.

References:

1. SharePoint connector | Databricks on AWS:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/sharepoint>

Question 27

A release pipeline deploys a reviewed Declarative Automation Bundle non-interactively. Authentication is configured, and the team has authorized deployment confirmation prompts to be accepted automatically.

Which flag should be added to databricks bundle deploy?

- A. `--auto-approve`, which accepts the deployment's confirmation prompts on the release pipeline's behalf instead of waiting for a person to answer them
- B. `--fail-on-active-runs`, which stops the non-interactive deployment whenever a job or pipeline defined in the bundle already has a run in progress
- C. `--output json`, which prints the deployment result in a machine-readable format that the release pipeline can then store in its own logs
- D. `--debug`, which prints detailed deployment logs that the release pipeline then reads in order to answer the confirmation prompts by itself

Correct answer: A

A. `--auto-approve`, which accepts the deployment's confirmation prompts on the release pipeline's behalf instead of waiting for a person to answer them

Explanation

Correct answer:

`--auto-approve`, which accepts the deployment's confirmation prompts on the release pipeline's behalf instead of waiting for a person to answer them

Why this is correct:

`--auto-approve` is the deployment flag for accepting confirmation prompts without an interactive operator. Because the reviewed bundle is intentionally deployed by automation and those confirmations are authorized, this flag matches the requested release behavior.

Why the other options are wrong:

`--fail-on-active-runs`, which stops the non-interactive deployment whenever a job or pipeline defined in the bundle already has a run in progress: This guard blocks deployment when managed resources are active; it does not answer confirmation prompts. It enforces a separate operational constraint.

`--output json`, which prints the deployment result in a machine-readable format that the release pipeline can then store in its own logs: JSON output changes the formatting of command results, not how the CLI handles confirmations. Machine-readable logs cannot approve prompts.

`--debug`, which prints detailed deployment logs that the release pipeline then reads in order to

answer the confirmation prompts by itself: Debug output increases diagnostic detail but does not respond to interactive prompts. The pipeline would still wait for human input.

References:

1. Develop Declarative Automation Bundles | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/work-tasks>

Question 28

A declarative pipeline defines **CONSTRAINT valid_qty EXPECT (qty > 0)** on a silver table, with no **ON VIOLATION** clause. During an update that otherwise succeeds, 40 incoming rows have qty = 0.

What happens to those 40 rows?

- A. The 40 rows are dropped from the silver table, and the violations are counted in the pipeline's data quality metrics for valid_qty
- B. The 40 rows are copied into a separate quarantine table, and the silver table keeps only the rows that pass the valid_qty check
- C. The whole table update fails before committing, and the pipeline reports valid_qty as the expectation that stopped the update
- D. The 40 rows stay in the silver table, and the violations are counted in the pipeline's data quality metrics for valid_qty

Correct answer: D

D. The 40 rows stay in the silver table, and the violations are counted in the pipeline's data quality metrics for valid_qty

Explanation

Correct answer:

The 40 rows stay in the silver table, and the violations are counted in the pipeline's data quality metrics for valid_qty

Why this is correct:

An expectation without an **ON VIOLATION** action is a warning: records remain in the target, while Databricks records violation counts in data-quality metrics. Dropping or failing requires an explicit action such as the corresponding expectation variant.

Why the other options are wrong:

The whole table update fails before committing, and the pipeline reports valid_qty as the expectation that stopped the update: The expectation's default is warning behavior, which records violations and retains rows. Failing the flow requires an explicitly selected failure action.

The 40 rows are copied into a separate quarantine table, and the silver table keeps only the rows that pass the valid_qty check: Expectations do not automatically create a quarantine table. Quarantine routing must be designed separately in the pipeline logic.

The 40 rows are dropped from the silver table, and the violations are counted in the pipeline's data quality metrics for valid_qty: Dropping violations requires an explicit drop action. With no violation clause, the warning behavior retains these rows and reports their counts.

References:

1. Expectations | Databricks Python reference:

<https://learn.microsoft.com/en-us/azure/databricks/ltp/developer/ltp-python-ref-expectations>

Question 29

A team needs a new namespace called reporting inside the existing corp catalog to hold several tables and views. The caller already has the CREATE SCHEMA privilege on corp.

Which statement creates the namespace the team needs?

- A. CREATE SCHEMA corp.reporting, which creates reporting inside the corp catalog as the second level of the three-level catalog.schema.object name
- B. CREATE CATALOG corp.reporting, which creates a child catalog named reporting beneath the corp catalog so that it can hold the team's tables and views
- C. CREATE VIEW corp.reporting, which creates a view in the corp catalog whose name then acts as the namespace for the team's other tables and views
- D. CREATE TABLE corp.reporting, which creates a table named reporting in the corp catalog that the team's later tables and views are then nested under

Correct answer: A

A. CREATE SCHEMA corp.reporting, which creates reporting inside the corp catalog as the second level of the three-level catalog.schema.object name

Explanation

Correct answer:

CREATE SCHEMA corp.reporting, which creates reporting inside the corp catalog as the second level of the three-level catalog.schema.object name

Why this is correct:

Unity Catalog names objects as catalog.schema.object. CREATE SCHEMA corp.reporting adds reporting as the namespace beneath the existing catalog; tables and views can then be created inside that schema.

Why the other options are wrong:

CREATE TABLE corp.reporting, which creates a table named reporting in the corp catalog that the team's later tables and views are then nested under: A table is a leaf data object, not a container namespace. Creating it would not allow other objects to be nested beneath it.

CREATE CATALOG corp.reporting, which creates a child catalog named reporting beneath the corp catalog so that it can hold the team's tables and views: Catalogs are top-level containers, not children of other catalogs. The required second-level namespace is a schema.

CREATE VIEW corp.reporting, which creates a view in the corp catalog whose name then acts as the namespace for the team's other tables and views: A view is also a data object within a schema; its name cannot serve as a namespace for separately created tables and views.

References:

1. CREATE SCHEMA | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-create-schema>

Question 30

A job has three upstream checks. Two succeeded and one failed. All three have completed, and none was excluded or canceled. A downstream task depends on all three checks and uses None failed.

What is the downstream task's expected state?

- A. It is marked Excluded, because None failed is treated as a failure-handling branch as soon as any of its three dependencies has failed
- B. It starts right away, because the job first retries the failed check automatically and then runs the downstream task afterward
- C. It runs, because the None failed condition needs only one of its three dependencies to have succeeded before the downstream task can start
- D. It is marked Upstream failed, because the None failed condition requires that none of its three dependencies finished in a failed state

Correct answer: D

D. It is marked Upstream failed, because the None failed condition requires that none of its three dependencies finished in a failed state

Explanation

Correct answer:

It is marked Upstream failed, because the None failed condition requires that none of its three dependencies finished in a failed state

Why this is correct:

None failed requires that no dependency finished in a failed state, so the completed failed check prevents the task from running. With all dependencies finished and none excluded or canceled, the task is marked upstream failed.

Why the other options are wrong:

It runs, because the None failed condition needs only one of its three dependencies to have succeeded before the downstream task can start: That reasoning confuses None failed with an any-success condition. One successful dependency cannot compensate for another dependency's failure under this run condition.

It is marked Excluded, because None failed is treated as a failure-handling branch as soon as any of its three dependencies has failed: Excluded is a distinct task state used when a task is skipped by workflow logic; a dependency failure under this condition produces upstream-failed status.

It starts right away, because the job first retries the failed check automatically and then runs the

downstream task afterward: A downstream condition does not launch or configure retries for a completed upstream task. Here every check is done, so evaluate their final states directly.

References:

1. Control the flow of tasks within Lakeflow Jobs | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/control-flow>

Question 31

A job must run an existing Lakeflow Spark Declarative Pipeline, then a reporting notebook that must wait for a successful pipeline update. The pipeline update must appear as a native task in the job graph.

Which task configuration should the job use?

- A. A Python wheel task that packages the reporting notebook, with no task anywhere in the job that references the existing pipeline
- B. A Run Job task that triggers the reporting job, while the pipeline keeps refreshing on its own separate schedule outside this job
- C. A SQL query task that reads the silver table, followed by a notebook task for the report that has no dependency on that SQL query task at all
- D. A Pipeline task that references the existing pipeline, followed by a report notebook task that runs only after the pipeline task succeeds

Correct answer: D

D. A Pipeline task that references the existing pipeline, followed by a report notebook task that runs only after the pipeline task succeeds

Explanation

Correct answer:

A Pipeline task that references the existing pipeline, followed by a report notebook task that runs only after the pipeline task succeeds

Why this is correct:

A Pipeline task makes the existing pipeline update a first-class node in the job graph. Making the report notebook depend on that task with success required ensures the report starts only after the pipeline update succeeds, in the intended order.

Why the other options are wrong:

A SQL query task that reads the silver table, followed by a notebook task for the report that has no dependency on that SQL query task at all: The pipeline update is missing from this graph, and the absent dependency edge leaves report timing uncontrolled. A table-reading query is not a substitute for the required pipeline task.

A Python wheel task that packages the reporting notebook, with no task anywhere in the job that references the existing pipeline: Packaging the report as a wheel changes how code is delivered, but it still does not schedule the existing pipeline as a task. The required upstream update is absent.

A Run Job task that triggers the reporting job, while the pipeline keeps refreshing on its own

separate schedule outside this job: A separate schedule has no dependency edge tying pipeline success to report start. Invoking a second job also fails to represent the pipeline update as a node here.

References:

1. Run pipelines in a workflow: <https://docs.databricks.com/aws/en/lp/workflows>

Question 32

A job defines the parameter region = west, and its notebook task also sets region = east. An operator starts a run that overrides the job parameter region to north. No task-value substitutions are involved.

Which region value is passed to the notebook?

- A. east, because the value set on the task takes precedence over any job parameter that has the same name when the run starts
- B. north, because the run-time job parameter value overrides both the job's default and the task value with the same name
- C. west, because the job's configured default takes precedence over any value that the operator supplies when the run is started
- D. No value, because the run fails validation whenever a job parameter and a task parameter are configured with the same name

Correct answer: B

B. north, because the run-time job parameter value overrides both the job's default and the task value with the same name

Explanation

Correct answer:

north, because the run-time job parameter value overrides both the job's default and the task value with the same name

Why this is correct:

Job run parameters override the configured job defaults and are pushed down to tasks. When names overlap, the job-level parameter takes precedence over a task's base value, so the notebook receives the run override north.

Why the other options are wrong:

east, because the value set on the task takes precedence over any job parameter that has the same name when the run starts: The task's configured value is only a lower-priority default when a job parameter shares its name. Runtime-supplied job values take precedence.

west, because the job's configured default takes precedence over any value that the operator supplies when the run is started: A job default is a fallback; the operator's run-time value intentionally replaces it for that run. Defaults do not block valid overrides.

No value, because the run fails validation whenever a job parameter and a task parameter are configured with the same name: A shared parameter name is supported and has defined precedence.

It does not make the job invalid; the higher-precedence job value is used.

References:

1. Access parameter values from a task | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/parameter-use>

Question 33

Every route value in DataFrame `df` holds two airport codes joined by a hyphen, such as `YYZ-YVR`. The engineer needs a new origin column containing the first code while keeping route. Assume `pyspark.sql.functions` is imported as `F`.

Which approach meets the requirement?

- A. Select `F.explode` over `F.split(route, '-')` aliased as `origin`, which produces a separate output row for each code in every route value
- B. Add `origin` with `withColumn`, taking the element at index 0 of `F.split(route, '-')`, and keep the route column in the DataFrame exactly as it is today
- C. Rename `route` to `origin` with `withColumnRenamed`, which keeps the full two-code value under the new column name and drops the old name
- D. Add `origin` with `withColumn`, taking the element at index 1 of `F.split(route, '-')`, and keep the route column in the DataFrame as it is

Correct answer: B

B. Add `origin` with `withColumn`, taking the element at index 0 of `F.split(route, '-')`, and keep the route column in the DataFrame exactly as it is today

Explanation

Correct answer:

Add `origin` with `withColumn`, taking the element at index 0 of `F.split(route, '-')`, and keep the route column in the DataFrame exactly as it is today

Why this is correct:

`split` returns an array in source order, so element zero is the origin in the stated `YYZ-YVR` format. `withColumn` adds or replaces one named column while retaining the rest of the DataFrame, including the original route value.

Why the other options are wrong:

Rename route to origin with `withColumnRenamed`, which keeps the full two-code value under the new column name and drops the old name: Renaming changes the column name, not the stored value. It would move the whole route string under `origin` and remove the field the prompt says to preserve.

Select `F.explode` over `F.split(route, '-')` aliased as `origin`, which produces a separate output row for each code in every route value: `explode` is a row generator: it turns array elements into separate rows. That changes row cardinality instead of adding one scalar origin value to each existing row.

Add origin with `withColumn`, taking the element at index 1 of `F.split(route, '-')`, and keep the route

column in the DataFrame as it is: Array position one is the second airport code, the destination under the format given. The origin is the first token, at zero-based index zero.

References:

1. **pyspark.sql.DataFrame.withColumn:** <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.withColumn.html>

Question 34

A For each task runs the same notebook once per store for ten independent store identifiers. The source system allows at most two simultaneous requests, and iterations should overlap as much as that limit permits.

Which configuration should be selected?

- A.** Set the job's maximum concurrent runs to ten, and leave the iteration concurrency of the For each task at its default value of one
- B.** Replace the For each task with one notebook task that sends the requests for all ten stores to the source system at the same time
- C.** Set the For each task's concurrency to ten, and reduce how often the job's schedule starts a new run to protect the source system
- D.** Set the For each task's concurrency to two, and pass the current store identifier to the nested notebook task as its parameter

Correct answer: D

D. Set the For each task's concurrency to two, and pass the current store identifier to the nested notebook task as its parameter

Explanation

Correct answer:

Set the For each task's concurrency to two, and pass the current store identifier to the nested notebook task as its parameter

Why this is correct:

For each task concurrency limits how many iterations of that task run simultaneously. Setting it to two respects the source system's cap while allowing two independent stores to overlap, and the nested task receives the current iteration's store parameter.

Why the other options are wrong:

Set the job's maximum concurrent runs to ten, and leave the iteration concurrency of the For each task at its default value of one: Maximum concurrent job runs governs separate job runs, not parallel iterations within one For each task. Leaving iteration concurrency at one serializes these stores.

Set the For each task's concurrency to ten, and reduce how often the job's schedule starts a new run to protect the source system: Concurrency ten allows ten simultaneous requests and exceeds the external system's limit. Slowing the schedule does not constrain overlap inside this single run.

Replace the For each task with one notebook task that sends the requests for all ten stores to the source system at the same time: Launching all ten requests together violates the stated cap and

discards the For each task's controlled per-iteration parameterization.

References:

1. For each task configuration | Declarative Automation Bundles:

<https://docs.databricks.com/gcp/en/dev-tools/bundles/job-task-types>

Question 35

During a two-week migration, a daily scheduled job must not start runs. Its cron expression, time zone and task settings must stay unchanged, and the same schedule must apply again afterward.

What should the team change?

- A. Delete the schedule after noting its cron expression, then recreate the same trigger by hand once the storage migration is done
- B. Set the schedule's trigger status to paused, then unpause it after the migration to resume the same cron expression and time zone
- C. Remove every task from the job so that each scheduled run has nothing to execute, keeping the cron expression and time zone for later
- D. Switch the job's trigger type to file arrival on a path that never receives files, keeping the cron expression and time zone stored for later use

Correct answer: B

- B. Set the schedule's trigger status to paused, then unpause it after the migration to resume the same cron expression and time zone

Explanation

Correct answer:

Set the schedule's trigger status to paused, then unpause it after the migration to resume the same cron expression and time zone

Why this is correct:

Pausing a trigger prevents it from starting new runs while preserving the configured schedule. Resuming it reactivates the same cron timing and time zone, which satisfies the temporary freeze without reconstructing job configuration.

Why the other options are wrong:

Switch the job's trigger type to file arrival on a path that never receives files, keeping the cron expression and time zone stored for later use: Replacing a schedule with a file trigger changes the job's trigger type and discards the required daily behavior while paused.

Remove every task from the job so that each scheduled run has nothing to execute, keeping the cron expression and time zone for later: Removing tasks leaves an altered, potentially invalid workflow and does not pause its trigger. It also requires restoring the tasks afterward.

Delete the schedule after noting its cron expression, then recreate the same trigger by hand once the storage migration is done: Deleting and recreating the schedule creates avoidable opportunities to lose the exact cron expression or time-zone setting; pause and resume are built for this temporary state.

References:

1. Run jobs on a schedule | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/scheduled>

Question 36

An ABAC column-mask policy must match columns tagged sensitivity=restricted but not those tagged sensitivity=public. All prerequisites are met.

```
CREATE POLICY restrict_text ON SCHEMA corp.silver  
COLUMN MASK corp.security.redact_text  
TO analysts  
FOR TABLES  
_____  
ON COLUMN secret_col;
```

Which clause completes the policy?

- A. MATCH COLUMNS has_tag_value('sensitivity', 'public') AS secret_col
- B. MATCH COLUMNS has_tag('sensitivity') AND has_tag('restricted') AS secret_col
- C. MATCH COLUMNS has_tag_value('sensitivity', 'restricted') AS secret_col
- D. MATCH COLUMNS has_tag_value('restricted', 'sensitivity') AS secret_col

Correct answer: C

C. MATCH COLUMNS has_tag_value('sensitivity', 'restricted') AS secret_col

Explanation

Correct answer:

MATCH COLUMNS has_tag_value('sensitivity', 'restricted') AS secret_col

Why this is correct:

The tag predicate names the governed tag key first and its required value second: sensitivity must equal restricted. The policy then selects tagged columns and applies its mask only to the matched secret_col binding.

Why the other options are wrong:

MATCH COLUMNS has_tag_value('restricted', 'sensitivity') AS secret_col: The helper arguments are reversed, so the predicate checks for a tag named restricted with value sensitivity. It will not match the tag specified in the requirement.

MATCH COLUMNS has_tag('sensitivity') AND has_tag('restricted') AS secret_col: has_tag tests tag presence and does not compare the tag's value to restricted. A separate value-aware predicate is

needed to exclude public-tagged columns.

MATCH COLUMNS has_tag_value('sensitivity', 'public') AS secret_col: This predicate selects public-tagged columns, the opposite of the requested restricted-only scope.

References:

- 1. Core concepts for attribute-based access control (ABAC) | Databricks on Google Cloud:**
<https://docs.databricks.com/gcp/en/data-governance/unity-catalog/abac/core-concepts>

Question 37

One version-controlled bundle is promoted from development to production. Pull requests must pass configuration validation, approved merges must deploy the production target, and production jobs must never be recreated by hand. Workspace and catalog settings differ per environment.

Which workflow best meets these requirements?

- A. Copy the bundle's resources between workspaces after pull requests merge, run smoke tests in each environment, and treat passing tests as the validation
- B. Deploy from developer notebooks, record the notebook revisions, and edit the production job settings by hand after validation and review
- C. Validate the bundle in pull requests, merge the approved changes, and deploy the prod target with its environment-specific overrides
- D. Deploy the prod target first from the approved pull request, validate the development target afterward, and fix mismatched resources by hand

Correct answer: C

C. Validate the bundle in pull requests, merge the approved changes, and deploy the prod target with its environment-specific overrides

Explanation

Correct answer:

Validate the bundle in pull requests, merge the approved changes, and deploy the prod target with its environment-specific overrides

Why this is correct:

Declarative Automation Bundles provide version-controlled configuration for jobs and pipelines, with validation before deployment and target-specific overrides. Merging reviewed changes and deploying the production target keeps resources managed as code instead of recreating them manually.

Why the other options are wrong:

Deploy from developer notebooks, record the notebook revisions, and edit the production job settings by hand after validation and review: Manual production edits drift from the reviewed source of truth and can be overwritten by later deployments. Notebook revisions do not validate the bundle's resource configuration.

Deploy the prod target first from the approved pull request, validate the development target afterward, and fix mismatched resources by hand: Deploying before review and validating only afterward reverses the release control. It also leaves mismatches to manual repair, defeating repeatable promotion.

Copy the bundle's resources between workspaces after pull requests merge, run smoke tests in each environment, and treat passing tests as the validation: Manual copying breaks the single versioned deployment path and can create workspace drift. Passing smoke tests does not validate bundle resource schemas.

References:

1. CI/CD on Databricks | Databricks on AWS: <https://docs.databricks.com/aws/en/dev-tools/ci-cd>

Question 38

A pipeline streaming table must incrementally process new rows from bronze_scans, an append-only table with no updates or deletes.

```
CREATE OR REFRESH STREAMING TABLE silver_scans AS  
SELECT scan_id, scanned_at  
FROM _____;
```

How should the blank reference the bronze source?

- A.** Use a materialized view built over bronze_scans, which the streaming table then reads for the rows that were added since the view's last refresh
- B.** Use bronze_scans on its own, relying on the streaming table definition to turn a plain table reference into an incremental read of new rows
- C.** Use table_changes('bronze_scans', 0), reading the bronze table's change data feed from its first version each time the pipeline runs an update
- D.** Use STREAM(bronze_scans), reading only the rows that were appended to the bronze table since the previous pipeline update was committed

Correct answer: D

D. Use STREAM(bronze_scans), reading only the rows that were appended to the bronze table since the previous pipeline update was committed

Explanation

Correct answer:

Use STREAM(bronze_scans), reading only the rows that were appended to the bronze table since the previous pipeline update was committed

Why this is correct:

STREAM(bronze_scans) gives the streaming query incremental semantics: its first update establishes the source position, and later updates consume newly appended rows from that point. The append-only premise is the key because no change-data-feed logic is needed to represent updates or deletes.

Why the other options are wrong:

Use table_changes('bronze_scans', 0), reading the bronze table's change data feed from its first version each time the pipeline runs an update: Change data feed is intended for consuming row-level changes, especially when updates and deletes matter. Reading from version zero on every refresh also ignores the simpler append-only streaming-table source pattern.

Use bronze_scans on its own, relying on the streaming table definition to turn a plain table

reference into an incremental read of new rows: A bare table reference is a batch relation; it does not become a streaming source merely because the destination is a streaming table. The source expression itself needs streaming semantics.

Use a materialized view built over bronze_scans, which the streaming table then reads for the rows that were added since the view's last refresh: A materialized view persists query results and refreshes them as a view; it is not a streaming source that tracks the source table's appended rows between pipeline updates.

References:

- 1. Use standalone streaming tables:** <https://docs.databricks.com/aws/en/ldp/dbsql/streaming>

Question 39

The offers table has a discounts column of type ARRAY<INT>. An engineer wants an array containing only values greater than ten, preserving their original order. The row count must not change.

Which SQL expression implements this transformation?

- A. EXPLODE(discounts), which returns one row per discount value so that values above ten can be filtered out in a later step
- B. FILTER(discounts, x -> x > 10), which returns a new array per row that keeps only the values above ten in their original order
- C. ARRAY_MAX(discounts), which returns the largest value in each array whenever that value is greater than ten for the row
- D. TRANSFORM(discounts, x -> x > 10), which returns an array holding the values greater than ten in their original order

Correct answer: B

B. FILTER(discounts, x -> x > 10), which returns a new array per row that keeps only the values above ten in their original order

Explanation

Correct answer:

FILTER(discounts, x -> x > 10), which returns a new array per row that keeps only the values above ten in their original order

Why this is correct:

The SQL FILTER higher-order function evaluates its lambda for each array element and returns only elements whose predicate is true. It preserves the surviving values' order inside each row's array and does not explode rows.

Why the other options are wrong:

EXPLODE(discounts), which returns one row per discount value so that values above ten can be filtered out in a later step: Turning elements into separate rows violates the explicit row-count constraint. Keep the array as a single value and filter its contents instead.

TRANSFORM(discounts, x -> x > 10), which returns an array holding the values greater than ten in their original order: TRANSFORM maps each element to a result; this lambda returns Boolean values, producing an array of true/false flags rather than the retained integers.

ARRAY_MAX(discounts), which returns the largest value in each array whenever that value is greater than ten for the row: ARRAY_MAX returns a single maximum value and discards the remaining

qualifying elements. The requirement is to retain every value above ten.

References:

1. filter function | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/functions/filter>

Question 40

A new Gold object must persist total revenue per region from corp.silver.sales, support later REFRESH operations, and let analysts in other sessions query the stored result. A supported SQL warehouse and permissions are available.

Which statement creates the required object?

- A. `CREATE TEMPORARY VIEW region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;`
- B. `CREATE VIEW corp.gold.region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;`
- C. `CREATE MATERIALIZED VIEW corp.gold.region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;`
- D. `CREATE TABLE corp.gold.region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;`

Correct answer: C

C. `CREATE MATERIALIZED VIEW corp.gold.region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;`

Explanation

Correct answer:

`CREATE MATERIALIZED VIEW corp.gold.region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;`

Why this is correct:

A materialized view stores the result of its defining query and supports refreshes to update that result. It provides a named, queryable persisted aggregate across sessions, unlike an ordinary view that recomputes from its source when queried.

Why the other options are wrong:

CREATE VIEW corp.gold.region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;; A standard view stores a query definition, not a persisted aggregate result with a refresh lifecycle. Analysts would cause the aggregate to be recomputed from the source at query time.

CREATE TABLE corp.gold.region_revenue AS SELECT region, SUM(revenue) AS total FROM

corp.silver.sales GROUP BY region;; A CTAS table persists rows but is not a materialized view with its refresh lifecycle. The requested object type is explicit in the need to issue refreshes.

CREATE TEMPORARY VIEW region_revenue AS SELECT region, SUM(revenue) AS total FROM corp.silver.sales GROUP BY region;; A temporary view is session-scoped and stores only a definition. Other sessions cannot rely on it as a persistent Gold object.

References:

1. Use standalone materialized views: <https://docs.databricks.com/aws/en/ldp/dbsql/materialized>

Question 41

An existing Auto Loader pipeline uses classic file notification mode with a dedicated queue, rather than managed file events. The required cloud notification resources and permissions have been provisioned.

```
.option("_____", "true")
```

Which option name belongs in the blank?

- A. `cloudFiles.includeExistingFiles`, which lets the existing pipeline read new files from the queue's events after loading the files already in the path once
- B. `cloudFiles.useManagedFileEvents`, which lets the existing pipeline discover new files through the file event service that runs on the external location
- C. `cloudFiles.inferColumnTypes`, which lets the existing Auto Loader pipeline discover new files through the provisioned queue and infer their column types
- D. `cloudFiles.useNotifications`, which lets the existing pipeline discover new files through the provisioned queue that receives the storage events

Correct answer: D

D. `cloudFiles.useNotifications`, which lets the existing pipeline discover new files through the provisioned queue that receives the storage events

Explanation

Correct answer:

`cloudFiles.useNotifications`, which lets the existing pipeline discover new files through the provisioned queue that receives the storage events

Why this is correct:

In classic notification mode, `cloudFiles.useNotifications=true` tells Auto Loader to discover files through the provisioned cloud event queue. Since the scenario excludes managed file events, this is the matching discovery configuration.

Why the other options are wrong:

`cloudFiles.inferColumnTypes`, which lets the existing Auto Loader pipeline discover new files through the provisioned queue and infer their column types: Schema type inference affects column types, not how new files are discovered. It does not connect the stream to the notification queue.

`cloudFiles.includeExistingFiles`, which lets the existing pipeline read new files from the queue's events after loading the files already in the path once: `includeExistingFiles` controls whether existing files are ingested when a stream first starts. It is not the switch for using event notifications.

cloudFiles.useManagedFileEvents, which lets the existing pipeline discover new files through the **file event service that runs on the external location**: Managed file events use the external location's managed event service, a different mode from the explicitly provisioned classic queue.

References:

1. Auto Loader FAQ | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/ingestion/cloud-object-storage/auto-loader/faq>

Question 42

A cluster fails at startup: driver logs show heap exhaustion while loading required libraries, and workers start normally. The same setup succeeds on a driver with more memory. The library set must stay unchanged.

Which change most directly addresses the observed failure?

- A. Select a driver instance type with more memory, keeping the same libraries and initialization settings for the cluster's next start
- B. Select worker instance types with more memory, keeping the current driver type, the libraries and the initialization settings as they are
- C. Reinstall the same library versions on the existing driver, clearing any partly loaded packages before the cluster starts again
- D. Raise the cluster's startup timeout for initialization, keeping the same driver instance type, memory and library set for its next start

Correct answer: A

A. Select a driver instance type with more memory, keeping the same libraries and initialization settings for the cluster's next start

Explanation

Correct answer:

Select a driver instance type with more memory, keeping the same libraries and initialization settings for the cluster's next start

Why this is correct:

The driver process loads and coordinates the required libraries, and its heap is the component reported as exhausted. Choosing a driver with more memory targets the failing resource; worker capacity cannot fix driver-side startup memory pressure.

Why the other options are wrong:

Select worker instance types with more memory, keeping the current driver type, the libraries and the initialization settings as they are: Workers run executors, but the logs show the driver fails during library loading. Increasing worker memory leaves the identified bottleneck unchanged.

Raise the cluster's startup timeout for initialization, keeping the same driver instance type, memory and library set for its next start: A longer timeout helps a slow process finish, not a process that runs out of heap. More time does not create additional memory.

Reinstall the same library versions on the existing driver, clearing any partly loaded packages before the cluster starts again: Reinstalling the unchanged libraries on the same undersized driver

repeats the condition described. The successful larger-driver run points to memory sizing.

References:

1. Compute configuration reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/compute/configure>

Question 43

The **STRING** column channel contains values such as **web**, **store**, and **NULL**. An engineer must replace only null channel values with unknown while preserving every other column and row. Assume `pyspark.sql.functions` is imported as `F`.

Which approach meets this requirement?

- A. Call `df.na.fill('unknown')` without a subset, which replaces the nulls in every string column of the DataFrame rather than in the channel column alone
- B. Call `df.na.fill` with a dictionary that maps only the channel column to unknown, leaving the nulls in every other column exactly as they are
- C. Call `df.withColumn` to set channel to `F.lit('unknown')`, which writes that literal into the channel column on every row of the DataFrame
- D. Call `df.na.drop` with subset set to channel, which removes each row whose channel value is null before the DataFrame is written out

Correct answer: B

B. Call `df.na.fill` with a dictionary that maps only the channel column to unknown, leaving the nulls in every other column exactly as they are

Explanation

Correct answer:

Call `df.na.fill` with a dictionary that maps only the channel column to unknown, leaving the nulls in every other column exactly as they are

Why this is correct:

The dictionary form of `na.fill` maps replacement values to named columns, so only channel is targeted. Because fill returns a new DataFrame, the input remains untouched, and nulls in unrelated columns are not part of the replacement map.

Why the other options are wrong:

Call `df.withColumn` to set channel to `F.lit('unknown')`, which writes that literal into the channel column on every row of the DataFrame: A literal replacement overwrites existing values as well as nulls, erasing web, store, and any other channel value. Conditional null handling is required to preserve those categories.

Call `df.na.drop` with subset set to channel, which removes each row whose channel value is null before the DataFrame is written out: drop removes affected records instead of repairing their channel field. That violates the requirement to preserve every row, even though it removes the missing value.

Call `df.na.fill('unknown')` without a subset, which replaces the nulls in every string column of the

DataFrame rather than in the channel column alone: A fill applied broadly to string columns changes unrelated nulls as well. The requirement is column-specific, so keep the replacement scoped to channel.

References:

1. fill (DataFrameNaFunctions) s:

<https://docs.databricks.com/aws/en/pyspark/reference/classes/dataframenafunctions/fill>

Question 44

The left table has keys 1, 2, and 3; the right table has keys 2, 2, and 4, with no nulls. An engineer inner joins them on key equality, with no DISTINCT or aggregation.

How many rows does the join return?

- A. Three rows, because an inner join keeps every row from the left table and fills the columns of unmatched rows with null values
- B. Four rows, because the output keeps all three right-side rows and then adds one unmatched key from the left side of the join
- C. One row, because an inner join returns each matching key a single time even when the right table repeats that key in several rows
- D. Two rows, because left key 2 pairs with each of the two right rows holding key 2 and no other key has a match on both sides

Correct answer: D

D. Two rows, because left key 2 pairs with each of the two right rows holding key 2 and no other key has a match on both sides

Explanation

Correct answer:

Two rows, because left key 2 pairs with each of the two right rows holding key 2 and no other key has a match on both sides

Why this is correct:

An inner join emits a row for every matching pair. Left key 2 matches two separate right-side rows, producing two pairs; keys 1 and 3 have no partner, and key 4 is not on the left.

Why the other options are wrong:

One row, because an inner join returns each matching key a single time even when the right table repeats that key in several rows: Joins do not deduplicate matching keys. Unless an explicit DISTINCT or aggregation collapses results, duplicate matches on one side multiply the output rows.

Three rows, because an inner join keeps every row from the left table and fills the columns of unmatched rows with null values: Preserving a left-side key without a match is outer-join behavior. An inner join emits only row pairs satisfying the equality condition.

Four rows, because the output keeps all three right-side rows and then adds one unmatched key from the left side of the join: Unmatched keys from either side are excluded by an inner join; they are not appended as extra rows. The result is determined by matching pairs only.

References:

1. JOIN | Spark SQL documentation:

<https://spark.apache.org/docs/latest/sql-ref-syntax-qry-select-join.html>

Question 45

Target and staged source each hold one row per non-null product_id. Matching prices must update without duplicating products on rerun, missing products must be added, and target-only products must stay unchanged.

```
MERGE INTO products AS t USING staged_products AS s
ON t.product_id = s.product_id
WHEN MATCHED THEN UPDATE SET t.price = s.price
_____;
```

Which clause completes this statement?

- A. WHEN MATCHED THEN INSERT (product_id, price) VALUES (s.product_id, s.price)
- B. WHEN NOT MATCHED BY SOURCE THEN INSERT (product_id, price) VALUES (t.product_id, t.price)
- C. WHEN NOT MATCHED THEN INSERT (product_id, price) VALUES (s.product_id, s.price)
- D. WHEN NOT MATCHED THEN UPDATE SET t.product_id = s.product_id, t.price = s.price

Correct answer: C

C. WHEN NOT MATCHED THEN INSERT (product_id, price) VALUES (s.product_id, s.price)

Explanation

Correct answer:

WHEN NOT MATCHED THEN INSERT (product_id, price) VALUES (s.product_id, s.price)

Why this is correct:

WHEN NOT MATCHED handles source rows with no target key match. Inserting the staged product_id and price adds new products; matched products are updated by the preceding clause, and target-only products are untouched.

Why the other options are wrong:

WHEN NOT MATCHED BY SOURCE THEN INSERT (product_id, price) VALUES (t.product_id, t.price): NOT MATCHED BY SOURCE identifies target rows absent from the staged input, not new staged products. Inserting target values there would also not satisfy the required source-side additions.

WHEN NOT MATCHED THEN UPDATE SET t.product_id = s.product_id, t.price = s.price: A source row that is not matched cannot be updated as a target match. WHEN NOT MATCHED requires an insert

action for the missing product.

WHEN MATCHED THEN INSERT (product_id, price) VALUES (s.product_id, s.price): The WHEN MATCHED branch already updates matching products. Inserting from that branch would duplicate matched keys rather than add only new products.

References:

1. Tutorial: Create and manage Delta Lake tables | Databricks on AWS:

<https://docs.databricks.com/aws/en/delta/tutorial>

Question 46

The `release_team` group must manage the bundle's reconcile job, with access defined in the job resource rather than applied manually after deployment.

```
resources:
  jobs:
    reconcile:
      name: reconcile
      ____:
        - group_name: release_team
          level: CAN_MANAGE
```

Which key belongs in the blank?

- A. `targets`, which lists `release_team` with the `CAN_MANAGE` level on the reconcile job for every deployment target that the bundle defines
- B. `variables`, which defines `release_team` and `CAN_MANAGE` as values that the bundle substitutes into the job during deployment
- C. `permissions`, which lists `release_team` with the `CAN_MANAGE` level to apply to the reconcile job each time the bundle deploys
- D. `run_as`, which sets `release_team` as the identity that runs the job and also gives the group `CAN_MANAGE` on that job

Correct answer: C

C. `permissions`, which lists `release_team` with the `CAN_MANAGE` level to apply to the reconcile job each time the bundle deploys

Explanation

Correct answer:

`permissions`, which lists `release_team` with the `CAN_MANAGE` level to apply to the reconcile job each time the bundle deploys

Why this is correct:

The resource-level `permissions` key declares grants for that job as part of the bundle definition. On deployment, the group permission is applied declaratively, avoiding a separate manual access change after each release.

Why the other options are wrong:

`targets`, which lists `release_team` with the `CAN_MANAGE` level on the reconcile job for every deployment target that the bundle defines: `targets` describes deployment environments and their overrides, not access-control grants on a specific job resource.

variables, which defines release_team and CAN_MANAGE as values that the bundle substitutes into the job during deployment: variables supplies configuration values for interpolation; it does not assign permissions to the job's principals.

run_as, which sets release_team as the identity that runs the job and also gives the group CAN_MANAGE on that job: run_as selects the identity under which job tasks execute. It is not the ACL that grants the group permission to manage the job.

References:

1. Set permissions for resources in Declarative Automation Bundles | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/permissions>

Question 47

A company runs three workspaces attached to one metastore. It wants a single place to define catalogs and schemas, grant table permissions once, and trace how source tables feed downstream dashboards across all three workspaces.

Which component provides this?

- A. Photon, the vectorized query engine, which applies the same table permissions and tracks the flow of data in each of the three attached workspaces
- B. Lakeflow Jobs, which records the permissions and data flow of the catalogs, schemas and tables its tasks use across the three workspaces
- C. Auto Loader, which stores each ingested table's permissions and data flow in its stream checkpoint for each of the three workspaces
- D. Unity Catalog, which governs those catalogs, schemas and grants centrally and records lineage across all three of the attached workspaces

Correct answer: D

D. Unity Catalog, which governs those catalogs, schemas and grants centrally and records lineage across all three of the attached workspaces

Explanation

Correct answer:

Unity Catalog, which governs those catalogs, schemas and grants centrally and records lineage across all three of the attached workspaces

Why this is correct:

Unity Catalog is the governance layer for catalogs, schemas, grants, and data discovery across workspaces attached to a metastore. Its lineage feature aggregates table flow across those workspaces, matching both the centralized administration and traceability requirements.

Why the other options are wrong:

Photon, the vectorized query engine, which applies the same table permissions and tracks the flow of data in each of the three attached workspaces: Photon accelerates query execution; it is not the catalog-level authorization or lineage service. Faster execution does not centrally define object grants.

Lakeflow Jobs, which records the permissions and data flow of the catalogs, schemas and tables its tasks use across the three workspaces: Task orchestration does not define centrally managed catalog permissions or lineage across workspaces. Those capabilities belong to the governance layer attached to the metastore.

Auto Loader, which stores each ingested table's permissions and data flow in its stream checkpoint for each of the three workspaces: Auto Loader discovers files and tracks ingestion progress in streaming state. It does not manage Unity Catalog privileges or cross-workspace table lineage.

References:

1. Lineage in Unity Catalog:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/data-lineage>

Question 48

A faulty UPDATE committed version 42 of the orders table. The team must compare current rows with version 41, then return the table to that state while keeping later history. VACUUM has not removed version 41's files.

Which TWO statements accomplish this?

- A. SELECT * FROM orders VERSION AS OF 41, which reads the table's rows exactly as they were at that earlier version of the table
- B. SELECT * FROM orders TIMESTAMP AS OF 41, which reads the table as it was at the moment that version 41 was committed to the log
- C. DESCRIBE HISTORY orders at version 41, which returns the full set of rows that the table held at that version for comparison
- D. RESTORE TABLE orders TO VERSION AS OF 41, which commits a new version whose contents match version 41 and keeps later history
- E. DROP TABLE orders followed by UNDROP at version 41, which recovers the table as it was before the faulty update was committed

Correct answers: A, D

- A. SELECT * FROM orders VERSION AS OF 41, which reads the table's rows exactly as they were at that earlier version of the table
- D. RESTORE TABLE orders TO VERSION AS OF 41, which commits a new version whose contents match version 41 and keeps later history

Explanation

Correct answer:

SELECT * FROM orders VERSION AS OF 41, which reads the table's rows exactly as they were at that earlier version of the table

RESTORE TABLE orders TO VERSION AS OF 41, which commits a new version whose contents match version 41 and keeps later history

Why this is correct:

VERSION AS OF 41 reads the exact snapshot for comparison, while RESTORE ... TO VERSION AS OF 41 commits a new current version matching that snapshot. Restore preserves subsequent history; it does not erase version 42 or the intervening log.

Why the other options are wrong:

SELECT * FROM orders TIMESTAMP AS OF 41, which reads the table as it was at the moment that version 41 was committed to the log: TIMESTAMP AS OF expects a timestamp value, not a version

integer. Version-based time travel must use the VERSION AS OF syntax.

DESCRIBE HISTORY orders at version 41, which returns the full set of rows that the table held at that version for comparison: History reports operation metadata, not the table's row contents. It cannot supply the historical dataset for comparing records.

DROP TABLE orders followed by UNDROP at version 41, which recovers the table as it was before the faulty update was committed: Dropping and undropping is a table-recovery workflow, not the correct version-level restoration operation. It also risks changing object lifecycle rather than creating the requested new table version.

References:

1. RESTORE | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/delta-restore>

Question 49

An analyst uploads a small CSV file through the workspace's create-table-from-file page. The preview infers `postal_code` as `INT`, which would strip leading zeros. The table has not been created yet.

What should the analyst do?

- A. Change the `postal_code` type to `STRING` in the schema preview, then create the table with that corrected `STRING` column type
- B. Choose a larger SQL warehouse for the upload page, which gives type inference a bigger sample of rows to infer `postal_code` as something other than `INT`
- C. Create the table with the types shown in the preview, then transfer its ownership to a data steward who reloads the postal codes as text
- D. Grant `USE SCHEMA` on the destination schema to the analyst, which lets the upload preview keep the inferred `postal_code` values as text in the new table

Correct answer: A

A. Change the `postal_code` type to `STRING` in the schema preview, then create the table with that corrected `STRING` column type

Explanation

Correct answer:

Change the `postal_code` type to `STRING` in the schema preview, then create the table with that corrected `STRING` column type

Why this is correct:

The schema preview is the point where inferred types can be reviewed and corrected before table creation. Setting `postal_code` to `STRING` preserves leading zeros because identifiers are not numeric quantities that should be normalized by integer conversion.

Why the other options are wrong:

Grant `USE SCHEMA` on the destination schema to the analyst, which lets the upload preview keep the inferred `postal_code` values as text in the new table: `USE SCHEMA` grants namespace access; it does not alter type inference or the preview schema. A permission change cannot preserve zeros stripped by an integer type.

Choose a larger SQL warehouse for the upload page, which gives type inference a bigger sample of rows to infer `postal_code` as something other than `INT`: A larger warehouse affects compute resources, not the semantic type selected for this column. More sampling could still infer numeric data and would not guarantee identifier-safe storage.

Create the table with the types shown in the preview, then transfer its ownership to a data steward who reloads the postal codes as text: Ownership transfer occurs after the table is already created with the lossy integer type. It does not restore leading zeros that were discarded during ingestion.

References:

1. Create or modify a table using file upload:

<https://docs.databricks.com/aws/en/ingestion/create-or-modify-table>

Question 50

A Databricks SQL session has ANSI mode enabled. An engineer evaluates the following expression without TRY_CAST or error handling:

```
SELECT CAST('invalid' AS INT);
```

What result should the engineer expect from this statement?

- A. The statement returns the value 0 as an INT, with ANSI mode substituting zero for any text that cannot be parsed as an integer
- B. Under ANSI mode the statement returns the original string unchanged as a STRING, and a cast warning is written to the query history
- C. Under ANSI mode the statement returns NULL as an INT result, with the cast error recorded in the session for the engineer to review later
- D. The statement fails with a malformed-input cast error under ANSI mode, and no result row is returned to the engineer's session

Correct answer: D

D. The statement fails with a malformed-input cast error under ANSI mode, and no result row is returned to the engineer's session

Explanation

Correct answer:

The statement fails with a malformed-input cast error under ANSI mode, and no result row is returned to the engineer's session

Why this is correct:

With ANSI mode enabled, an invalid string-to-integer cast raises a malformed-input error rather than silently returning a substitute. Since this expression has no TRY_CAST or error-handling wrapper, statement evaluation fails and produces no result row.

Why the other options are wrong:

Under ANSI mode the statement returns the original string unchanged as a STRING, and a cast warning is written to the query history: A cast does not preserve the original expression's STRING type after a failed conversion. ANSI mode surfaces the invalid input as an error instead of issuing a warning and returning it.

The statement returns the value 0 as an INT, with ANSI mode substituting zero for any text that cannot be parsed as an integer: ANSI casting does not choose zero as a fallback for malformed text. Substituting a default would require explicit logic in the query.

Under ANSI mode the statement returns NULL as an INT result, with the cast error recorded in the session for the engineer to review later: Returning null on invalid conversion is the tolerant behavior associated with TRY_CAST or disabled ANSI behavior in applicable contexts, not this stated ANSI cast.

References:

1. ANSI compliance in Databricks Runtime | Azure Databricks:

<https://learn.microsoft.com/en-us/azure/databricks/sql/language-manual/sql-ref-ansi-compliance>

Question 51

A deployment pipeline is about to update bundle-managed jobs and pipelines in production. Deployment must fail if any of those resources has an active run. Authentication and the prod target are configured.

Which command meets this requirement?

- A. databricks bundle validate with the prod target, which checks for active runs and configuration errors before any resource is updated
- B. databricks bundle deploy with the prod target and --fail-on-active-runs, which stops the deployment when any bundle resource has a run in progress
- C. databricks bundle run with the prod target, which makes the deployment step fail when the reconcile job still has an active run
- D. databricks bundle deploy with the prod target and --auto-approve, which accepts the prompts about resources that currently have active runs

Correct answer: B

B. databricks bundle deploy with the prod target and --fail-on-active-runs, which stops the deployment when any bundle resource has a run in progress

Explanation

Correct answer:

databricks bundle deploy with the prod target and --fail-on-active-runs, which stops the deployment when any bundle resource has a run in progress

Why this is correct:

bundle deploy --fail-on-active-runs makes deployment fail when a job or pipeline in the bundle currently has a run in progress. It enforces the release guard at deployment time while the explicit prod target directs the update to production.

Why the other options are wrong:

databricks bundle run with the prod target, which makes the deployment step fail when the reconcile job still has an active run: bundle run starts a configured resource; it is not the command that applies the bundle deployment guard. Running the job does not protect all managed resources during deployment.

databricks bundle validate with the prod target, which checks for active runs and configuration errors before any resource is updated: Schema validation is a preflight check; it does not perform deployment or reject an update because a managed run is active. The required guard belongs on deploy.

databricks bundle deploy with the prod target and --auto-approve, which accepts the prompts

about resources that currently have active runs: Prompt automation removes interactive pauses but does not inspect resource run states. This flag would accept deployment confirmations instead of stopping on active work.

References:

1. Databricks CLI commands | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/cli/commands>

Question 52

A valid streaming DataFrame named incoming is ready to be written. The query should remain active and attempt to start a new micro-batch every 45 seconds, subject to processing time.

```
incoming.writeStream._____.option(  
    "checkpointLocation", checkpoint_path  
) .toTable("ops.bronze.scans")
```

Which trigger expression completes the writer?

- A. trigger(availableNow=True), which processes all the data available when the query starts in micro-batches and then stops writing to the bronze scans table
- B. trigger(processingTime='0 seconds'), which starts each new micro-batch for the bronze scans table as soon as the previous one completes, while the query remains active
- C. trigger(processingTime='45 seconds'), which starts a new micro-batch for the bronze scans table at each 45-second interval while the query keeps running
- D. trigger(once=True), which processes one micro-batch for the bronze scans table every 45 seconds until an operator stops the stream by hand

Correct answer: C

C. trigger(processingTime='45 seconds'), which starts a new micro-batch for the bronze scans table at each 45-second interval while the query keeps running

Explanation

Correct answer:

trigger(processingTime='45 seconds'), which starts a new micro-batch for the bronze scans table at each 45-second interval while the query keeps running

Why this is correct:

The processing-time trigger keeps the streaming query active and asks it to start another micro-batch every 45 seconds, subject to the previous batch finishing. This is the requested periodic cadence rather than a finite one-time drain.

Why the other options are wrong:

trigger(processingTime='0 seconds'), which starts each new micro-batch for the bronze scans table as soon as the previous one completes, while the query remains active: A zero-second interval requests the next batch as soon as possible, not a 45-second cadence. It can continuously process available data while the query stays active.

trigger(availableNow=True), which processes all the data available when the query starts in

micro-batches and then stops writing to the bronze scans table: availableNow drains currently available input in one or more batches and then terminates. It is a bounded run, not a recurring active query.

trigger(once=True), which processes one micro-batch for the bronze scans table every 45 seconds until an operator stops the stream by hand: The one-shot trigger terminates after a batch, so no timer remains to start the next batch at the requested interval.

References:

1. trigger (DataStreamWriter) | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/pyspark/reference/classes/datastreamwriter/trigger>

Question 53

A job runs prepare first. Two independent validation tasks should then run in parallel, followed by publish only when both validations succeed.

Which dependency configuration expresses this workflow?

- A. Make prepare depend on both validation tasks, then make publish depend on prepare with the All succeeded run condition
- B. Chain the tasks as prepare, the first validation, the second validation and then publish, each depending on the one before
- C. Make both validation tasks and publish depend only on prepare so that all three run in parallel, each set to the All succeeded condition
- D. Make both validation tasks depend on prepare, and make publish depend on both validations with the All succeeded run condition

Correct answer: D

D. Make both validation tasks depend on prepare, and make publish depend on both validations with the All succeeded run condition

Explanation

Correct answer:

Make both validation tasks depend on prepare, and make publish depend on both validations with the All succeeded run condition

Why this is correct:

Dependencies define the execution graph: each validation waits for prepare, and publish waits for both validations. All succeeded makes publication conditional on both independent checks completing successfully.

Why the other options are wrong:

Make both validation tasks and publish depend only on prepare so that all three run in parallel, each set to the All succeeded condition: Publication becomes eligible immediately after preparation, so it can race ahead of both checks. Its dependency set must include both validation tasks.

Make prepare depend on both validation tasks, then make publish depend on prepare with the All succeeded run condition: This reverses the intended flow: prepare would wait for validations, which themselves are supposed to run after prepare. It also cannot express both validations as its prerequisites.

Chain the tasks as prepare, the first validation, the second validation and then publish, each depending on the one before: A chain serializes the two independent validations, adding unnecessary

ordering and allowing publish to depend indirectly on only the last validation's branch.

References:

1. Lakeflow Connect connector concepts | Microsoft Learn:

<https://learn.microsoft.com/en-us/azure/databricks/ingestion/lakeflow-connect/>

Question 54

An engineer runs the following statement after confirming that the target does not exist:

```
CREATE TABLE analytics.silver.completed_orders USING DELTA AS
SELECT order_id, CAST(amount AS DECIMAL(12,2)) AS amount
FROM analytics.bronze.orders
WHERE status = 'COMPLETE';
```

Which statement describes the new table?

- A. It stores the completed orders as Delta data, with column types taken from the query, including the amount column as DECIMAL(12,2)
- B. It stores only the query definition, rereads the bronze table, and applies the status filter each time that the table is queried
- C. It holds every bronze row, with CTAS copying the whole source table before the WHERE clause is applied to any later reads
- D. It inherits every constraint and grant from the bronze orders table, along with the selected rows and the amount column's DECIMAL type

Correct answer: A

- A. It stores the completed orders as Delta data, with column types taken from the query, including the amount column as DECIMAL(12,2)

Explanation

Correct answer:

It stores the completed orders as Delta data, with column types taken from the query, including the amount column as DECIMAL(12,2)

Why this is correct:

CTAS materializes the query output as a new Delta table. Its stored rows reflect the WHERE filter, and the query's selected expression defines the resulting column types, including the explicit decimal cast for amount.

Why the other options are wrong:

It stores only the query definition, rereads the bronze table, and applies the status filter each time that the table is queried: A query-only object would be a view, not the Delta table created by CREATE TABLE ... USING DELTA AS SELECT. The requested CTAS stores result data.

It holds every bronze row, with CTAS copying the whole source table before the WHERE clause is applied to any later reads: The WHERE clause is applied while evaluating the CTAS query, so incomplete or non-complete bronze rows are not copied into the new table.

It inherits every constraint and grant from the bronze orders table, along with the selected rows and the amount column's DECIMAL type: CTAS derives the new table schema from its query; it does not automatically inherit every source constraint or grant. Object permissions are governed separately.

References:

1. CREATE TABLE | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/sql/language-manual/sql-ref-syntax-ddl-create-table>

Question 55

A Delta table named `ops.bronze.scans` receives only appended records. An engineer needs to use it as the source of a Structured Streaming transformation rather than repeatedly reading its entire current snapshot.

Which approach creates the required streaming `DataFrame`?

- A. Use `spark.read.table` on the table, which returns a `DataFrame` that keeps track of the appended rows between query executions
- B. Use `spark.read` with the delta format on the table, which returns a `DataFrame` that picks up only the newly appended rows on each read
- C. Use `spark.table` on the table, which returns a `DataFrame` that reads only the latest appended rows each time that it is queried
- D. Use `spark.readStream.table` on the table, which returns a streaming `DataFrame` that processes the new rows as they are appended

Correct answer: D

D. Use `spark.readStream.table` on the table, which returns a streaming `DataFrame` that processes the new rows as they are appended

Explanation

Correct answer:

Use `spark.readStream.table` on the table, which returns a streaming `DataFrame` that processes the new rows as they are appended

Why this is correct:

`readStream.table` creates a streaming `DataFrame` over the named Delta table. Structured Streaming then consumes new appended data as the query advances, unlike `read` or `table`, which create batch `DataFrames` representing a snapshot.

Why the other options are wrong:

Use `spark.read` with the delta format on the table, which returns a `DataFrame` that picks up only the newly appended rows on each read: Batch reads produce a snapshot each time and retain no offset between calls. Incremental processing requires a streaming reader and checkpointed query.

Use `spark.read.table` on the table, which returns a `DataFrame` that keeps track of the appended rows between query executions: This reader still builds a batch `DataFrame` from the current snapshot. It has no streaming offset that advances as appends arrive.

Use `spark.table` on the table, which returns a `DataFrame` that reads only the latest appended rows each time that it is queried: Table resolution returns the current batch snapshot; it does not remember

which rows were previously processed or yield an incremental stream.

References:

1. Delta Lake table streaming reads and writes | Databricks on AWS:

<https://docs.databricks.com/aws/en/structured-streaming/delta-lake>